

0909 Baseline Report — Gemma-4-12B-it on FuriosaAI RNGD

작성일 2026-09-09 · 최종 갱신 2026-09-10 · 기준선 v0_baseline · 현재 공식 SOTA v204_submit (공식 채점 5.9032x, Arena 실측 6.229x) 상위 규칙은 RULES.md, 실험 기록은 RESULTS.md, 신기록 서사는 SOTA.md에 있다.

0. 이 문서를 읽는 법

대상 독자. 학부 수준의 선형대수(행렬곱)와 C/Python 정도의 프로그래밍 경험이 있는 사람. NPU, 텐서 컴파일러, LLM 추론 최적화를 몰라도 읽을 수 있게 썼다. Rust를 몰라도 된다 — 이 문서에 나오는 Rust 코드는 "읽는 법"을 §3에서 따로 가르친다.

구성.

절	내용	이미 아는 사람은
§1	프로젝트와 대회가 뭔지	훑고 넘어가기
§2	NPU가 어떻게 동작하는가 — 특히 §2.7 실물 비용 모델	§2.7은 보기
§3	furiosa-opt 코드 읽는 법 — 특히 §3.3 contraction 파이프라인	§3.3만 보기
§4	최적화 원시 카탈로그	여기가 핵심
§5~§8	대회 규칙, 코드 지도, 커널 해부, 측정 체계	필요할 때 찾아보기
§9	지금까지 뭘 했고 다음은 뭔지	여기가 핵심

§2.7이 이번 판에서 가장 크게 바뀐 곳이다. 이전 판이 "~290 B/cycle 벽에 도달했다"고 결론지었는데, 그 벽을 직접 재보니 **벽이 아니라 런 길이가 잘못된 지점**이었다(§2.7.5). 정정 과정 자체가 이 프로젝트에서 가장 값진 방법론이라 §2.7.4~2.7.7로 나눠 남겼다.

그리고 §2.7의 토대는 이것이다. 이 하드웨어의 실물 비용이 바이트가 아니라 **DMA 디스크립터 수**에 묶인다는 것 — 프로젝트 후반부의 모든 발견이 여기서 나온다. §9.5가 그 발견 과정이다.

§3.3도 크게 늘어난 부분이다. contraction이 왜 4단으로 쪼개지는지, 그 인자 하나를 바꾸면 왜 벡터 부하가 16배 달라지는지를 저장소의 실제 코드 두 덩어리로 따라간다. §4의 원시들이 결국 전부 "이 파이프라인의 어느 손잡이를 돌릴 것인가"의 문제이므로, §3.3을 건너뛰면 §4가 주문처럼 보인다.

1. 한눈에 보기

1.1 무엇을 하는 프로젝트인가

Google의 오픈 LLM **Gemma-4-12B-it**(120억 파라미터)을 **FuriosaAI RNGD**라는 AI 전용 칩 위에서 최대한 빠르게 돌리는 것이다. 이 저장소에는 이미 동작하는 구현이 통째로 들어 있다 — 모델 실행, 이미지·오디오 입력, HTTP 서버까지. 우리가 하는 일은 **그중 딱 세 개의 함수를 더 빠르게 고쳐 쓰는 것이다**.

대회 이름은 MOA @ MICRO 2026 Kernel Optimization Round이고, 잘하면 아테네에서 발표한다.

1.2 지금 어디까지 왔나

RNGD 실측 기준 (Arena job 15360, 3/3 정확도 PASS):

공식 채점 결과 (moa-submitter 46bc9eed, 2026-09-10 05:12 UTC, 3/3 PASS):

	sliding_project_qkv	sliding_attention_output	decoder_feedforward	기하평균
공식 baseline	250,514	404,633	3,703,473	1.000×
v204 공식 채점	101,132	56,845	317,445	—
speedup	2.48×	7.12×	11.67×	5.9032×

리더보드 팀 **Goat Chovy #1557**. 확정된 SOTA 계보: V0 → V7(2.43×) → V14(3.94×) → V50(4.93×) → V82(5.086×) → V165(5.758×) → **V204(5.903×**).

공식 draw가 나뻐다. 같은 코드의 Arena 실측(잡 14개)은 qkv 100.3k / attn_out **49.9k** / ffn 310.1k = **6.229**인데, 공식 채점에서 attn_out만 56,845로 나왔다(Arena 범위 47.9k~52.0k 밖). 코드 차이가 아니라 **cold 1회 draw의 분산**이다. 재제출만 해도 6.0대가 기대되지만 제출 예산은 하루 5회다.

세 커널이 만난 것은 벽이 아니었다

V165 시점에 세 커널이 전부 ~290 B/cycle에 모였다. 당시 이 문서는 그것을 **물리적 한계**라고 읽었다. **그 판단이 틀렸다.**

V197이 DMA 비용 곡선을 직접 실측하면서 정체가 드러났다 — **그건 벽이 아니라 "런 길이가 잘못된 지점의 값"이었다.**

런 길이	실효 속도
128 B	319 B/cycle
256 B	618 B/cycle ← 최적
512 B	595 B/cycle
2048 B	547 B/cycle

256 B 미만은 정확히 2× 벌점이고(128 B가 256 B의 1.94배), 길수록 좋다는 통념은 틀렸다(256 → 2048에서 13% 악화). 그리고 최적점 618 B/cycle은 스펙의 41%다 — 우리가 벽이라 불렀던 19%의 두 배가 넘는다.

자세한 건 §2.7이다. 이 정정이 이번 판에서 가장 중요한 내용이다.

여기까지 오는 길에 한 번 크게 넘어졌다

2026-09-09 첫 실측 전까지 이 프로젝트는 **makespan**(컴파일러가 짜놓은 계획표의 길이)만 보고 달렸고, 그 위에서 5.801×까지 올려놨었다. 첫 Arena 제출에서 **V15 이후 전 계보가 정확도 FAIL**로 드러났다. 원인은 성능 모델이 아니라 **틀린 코드**였다 — 자세한 건 §9.2.

살아남은 것을 추려 다시 세운 게 V50이고, 거기서 qkv의 진짜 병목을 찾아낸 것이 V165다(§9.5). 이 보고서의 성능 수치는 **전부 실측이고, 헤드라인은 공식 채점 결과다**.

오랫동안 **qkv가 유일한 병목이었다**. 1.29× → 1.34× → 1.64×로만 움직이며 다른 둘(7×, 10×)에 한참 못 미쳤다. 그 벽을 깬 것이 V165이고(2.37×), 깬 방법이 이 프로젝트에서 가장 중요한 발견이다 — **실물 비용은 바이트가 아니라 DMA 디스크립터 수에 묶인다**(§2.7, §9.5).

2. NPU가 어떻게 동작하는가 — 30분 속성 과정

2.1 CPU, GPU, 그리고 NPU

셋 다 곱셈과 덧셈을 하는 기계인데, "한 번에 몇 개를, 얼마나 자유롭게"가 다르다.

	한 번에 처리	잘하는 것	못하는 것
CPU	몇 개 (코어 수만큼)	뭐든지. 분기, 포인터, 조건문	대량 병렬
GPU	수천 개 (스레드)	같은 연산을 데이터만 바꿔 반복	스레드마다 다른 일
NPU (RNGD)	수십만 개	텐서 연산 하나	그 외 전부

NPU는 극단으로 간 특수 목적 기계다. 대신 그 하나를 아주 잘한다. RNGD는 BF16 기준 초당 256조 번 곱셈-덧셈 (256 TFLOPS)을 한다.

여기서 중요한 차이 하나. GPU에서 CUDA를 쓰면 "스레드 100만 개를 띄워라"라고 쓰고 스케줄링은 하드웨어가 알아서 한다. **RNGD에서는 그 배치를 프로그래머가 직접 쓴다**. 어떤 데이터를 어느 메모리 층에 놓을지, 어느 연산 유닛에 태울지, 몇 개씩 쪼갤지를 전부 코드로 지정한다. 그래서 이 대회가 성립한다 — 같은 수학인데 배치를 바꾸면 5배가 달라진다.

2.2 contraction이 뭔가 — 행렬곱의 일반화

RNGD의 유일한 원시 연산이 텐서 contraction이다. 이름이 낯설지 뭐지, 사실 이미 아는 것이다.

행렬 × 벡터를 생각해보자.

$$y_i = \sum_j W_{ij} \cdot x_j$$

여기서 벌어지는 일은 세 단계다.

- 1. W_{ij} 와 x_j 를 짝지어 곱한다 (같은 j 끼리)
- 2. j 에 대해 전부 더한다
- 3. 남은 i 가 결과의 축이 된다

"두 텐서를 공통 축에서 곱하고 그 축을 따라 더해서 없애는 것" — 이게 contraction이다. 사라지는 축(j)을 **접힌다(contracted)**고 한다. 행렬곱, 행렬-벡터곱, 내적, 배치 행렬곱, 컨볼루션이 전부 이 하나의 틀에 들어간다. 어느 축을 접느냐만 다르다.

내적	$a \cdot b$	$= \sum_i a_i b_i$	(i 를 접음, 남은 축 없음 $\rightarrow$ 스칼라)
행렬×벡터	$y = Wx$	$= \sum_j W_{ij} x_j$	(j 를 접음, i 가 남음)
행렬×행렬	$C = AB$	$= \sum_k A_{ik} B_{kj}$	(k 를 접음, $i \cdot j$ 가 남음)
어텐션 점수	$S = QK^T$	$= \sum_d Q_{hd} K_{td}$	(d 를 접음, $h \cdot t$ 가 남음)

GPU와의 차이가 여기 있다. GPU는 이 모든 걸 2D 행렬곱(GEMM)으로 바꿔서 처리한다. 4차원 텐서가 들어오면 reshape/transpose로 2D로 눌러 편 다음 GEMM을 부르고 다시 펴는데, 그 눌렀다 펴는 과정 자체가 메모리를 왕복하는 비용이다. **RNGD는 다차원 contraction을 그대로 실행한다.** 그래서 이름이 Tensor Contraction Processor(TCP)다.

우리 커널에서 실제로 접히는 축은 이렇다.

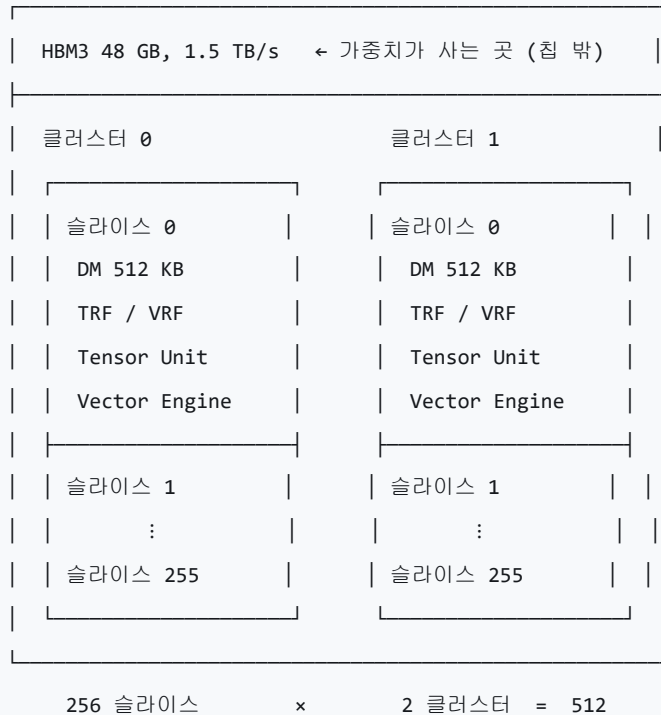
연산	식	접히는 축	크기
Q 투영	$q[o] = \sum_h Wq[o,h] \cdot x[h]$	H	3,840
K/V 투영	$k[o] = \sum_h Wk[o,h] \cdot x[h]$	H	3,840
O 투영	$y[h] = \sum_o Wo[h,o] \cdot \text{attn}[o]$	Qs	4,096
FFN up/gate	$u[l] = \sum_h Wu[l,h] \cdot x[h]$	H	3,840
FFN down	$y[h] = \sum_l Wd[h,l] \cdot g[l]$	L	15,360

전부 **행렬 × 벡터**다. 토큰 하나씩 처리하기 때문이다(\$2.6에서 다시 다룬다).

2.3 RNGD의 몸 — 512개의 작은 컴퓨터

칩을 안에서부터 밖으로 그리면 이렇다.

RNGD 칩 하나



슬라이스(slice)가 기본 작업 단위다. 각자 512 KB짜리 전용 메모리(DM)와 연산기를 갖는다. $512 \times 512 \text{ KB} = 256 \text{ MB}$ — 스펙시트의 온칩 SRAM 용량과 정확히 맞는다.

행렬 $\times$ 벡터를 512개로 나누는 방법은 자연스럽다. **출력 행을 나눠 갖는다.**

```
q[0..4095] = Wq[4096 × 3840] · x[3840]
```

슬라이스 0 → Wq의 0~7번 행 담당 → q[0..7] 계산

슬라이스 1 → Wq의 8~15번 행 담당 → q[8..15] 계산

:

슬라이스 511 → Wq의 4088~4095행 담당 → q[4088..4095] 계산

각 슬라이스는 **자기 행만** 갖고 있으면 되는데, **입력 x는 전부가 똑같이 필요하다**. 512개 슬라이스 전부에 3,840 개짜리 벡터를 복사해줘야 한다. 이 복사가 공짜가 아니고, 실제로 이 프로젝트에서 가장 먼저 잡은 병목이었다 (§9, V7).

함정 하나. v0_baseline의 코드는 `cluster = m![1 # 2]`로 되어 있었는데, 이게 "논리적으로 1개를 물리적으로 2개 자리에 복제"라는 뜻이다. 즉 **두 클러스터가 똑같은 계산을 하고 있었다** — 칩의 절반이 놀고 있었다. 이걸 고치는 게 v14_two_clusters다.

2.4 메모리 5층 구조

속도와 크기가 반비례하는 계단이다. 도서관에 비유하면 이렇다.

층	크기	비유	코드 타입
HBM	48 GB	서고 — 다 있지만 가지러 가야 함	HbmTensor
DM	512 KB × 512	내 책상 — 슬라이스마다 하나씩	DmTensor
TRF	작음	펼쳐놓은 책 — contraction 피연산자	TrfTensor
VRF	작음	손에 든 메모지 — 벡터 상수	VrfTensor
레지스터	—	지금 보는 줄	(직접 안 다룸)

데이터는 항상 이 계단을 타고 오르내린다.



계단을 몇 번 오르내리느냐가 곧 성능이다. 이 프로젝트 최적화의 절반은 "왕복 한 번 줄이기"였다.

2.5 세 명의 일꾼 — 실행 컨텍스트

RNGD 안에는 동시에 일할 수 있는 일꾼이 셋 있다. 코드에서 ctx.main, ctx.sub, ctx.tdma 로 부른다.

일꾼	코드	하는 일
MainContext	ctx.main	주력. contraction, 벡터 연산, 타입 변환, 슬라이스 간 데이터 이동
SubContext	ctx.sub	조수. 레지스터 파일에 미리 올려놓기 (to_trf , to_vrf)
DmaEngine	ctx.tdma	짐꾼. HBM ↔ DM, DM ↔ DM 이동

규칙 세 개만 기억하면 된다.

규칙 1 — 같은 일꾼에게 시킨 일은 순서대로 처리된다. ctx.main 에 A, B, C를 시키면 A→B→C다. 아무리 A와 B가 독립이어도 겹치지 않는다.

규칙 2 — 다른 일꾼끼리는 겹친다. 짐꾼이 가중치를 나르는 동안 주력이 계산할 수 있다. 단, 서로 같은 데이터를 건드리면(의존) 기다려야 한다.

규칙 3 — MainContext와 SubContext는 같은 파이프라인을 공유한다. 완전히 겹치지는 않는다. 최악의 경우 둘의 합, 잘되면 큰 쪽 시간에 수렴한다.

여기서 핵심 최적화 아이디어 하나가 바로 나온다.

한 일꾼이 96% 바쁘고 나머지가 놀고 있다면, 일을 옮기기만 해도 빨라진다.

v0_baseline 이 정확히 그 상태였다. MainContext 가 96%를 차지했고, 짐꾼(DMA)은 대부분 놀고 있었다.

2.6 왜 LLM 디코딩은 "느린 게 정상"인가 — 루프라인

이게 이 프로젝트에서 제일 중요한 개념이다. 천천히 가자.

LLM은 토큰을 하나씩 만든다. "안녕"을 만들고, 그걸 다시 입력에 넣어 "하세요"를 만든다. 순차적이라 건너뛸 수 없다. 그래서 한 번의 계산에 들어가는 입력은 **토큰 딱 하나 = 3,840개짜리 벡터 하나**다.

그런데 그 벡터 하나를 처리하려고 **가중치 행렬 전체를 메모리에서 읽어와야 한다**.

sliding_project_qkv 한 번 실행:

읽는 데이터: Q·K·V 가중치 30 MB

하는 계산: 31,500,000 번의 곱셈-덧셈

이 둘의 비율을 **산술 강도(arithmetic intensity)**라고 한다.

$$\text{산술 강도} = \frac{\text{연산 횟수}}{\text{읽은 바이트}} = \frac{2 \times 31.5\text{M}}{30\text{MB}} \approx 2.0 \text{ FLOP/byte}$$

바이트 하나 읽어와서 곱셈 두 번 하고 버린다. 이제 하드웨어 쪽 비율을 보자.

$$\text{RNGD의 균형점} = \frac{256 \text{ TFLOPS}}{1.5 \text{ TB/s}} \approx 170 \text{ FLOP/byte}$$

170이 필요한데 2를 준다. 85배 부족하다. 결론은 명확하다.

이 커널들은 계산이 느려서 느린 게 아니라, 데이터를 못 갖다 줘서 느리다. 연산기는 대부분의 시간을 놀면서 기다린다. 이런 상태를 **메모리 바운드(memory-bound)**라고 한다.

숫자로 확인해보자. 1 GHz니까 **1.5 TB/s = 사이클당 1,500 바이트**, **256 TFLOPS = 사이클당 128,000 MAC**이다.

	데이터를 다 읽는 데	계산을 다 하는 데
sliding_project_qkv	~21,000 cycle	~250 cycle
sliding_attention_output	~10,500 cycle	~125 cycle
decoder_feedforward	~66,000 cycle	~1,400 cycle

읽는 시간이 계산 시간의 50배 안팎이다. 그러니 "행렬곱을 빠르게" 하는 최적화는 여기서 의미가 거의 없다. 이미 연산기는 놀고 있다.

그런데 실제 **baseline**은 그보다도 훨씬 느렸다. (아래는 전부 RNGD 실측 cycle이다)

	이론 하한	V0_baseline 실측	배수	V82 실측	배수
sliding_project_qkv	~21,000	244,885	11.7배	144,863	6.9배
sliding_attention_output	~10,500	405,253	38.6배	57,239	5.5배
decoder_feedforward	~66,000	3,706,465	56.2배	348,994	5.3배

베이스라인은 이동 하한보다 12~56배 위에 있었다. **대역폭도 연산량도 아닌 제3의 무언가가 시간을 먹고 있었다는 뜻이다.** 그 뒤의 실험들이 한 일이 그걸 걷어내는 작업이었고, 지금은 셋 다 5~7배까지 내려왔다.

비유. 트럭으로 짐을 나르는데 왕복 시간(하한)이 1시간인데 실제로는 하루가 걸렸다. 도로가 막혀서가 아니라, 짐을 싣기 전에 창고에서 포장을 뜯었다 다시 싸고, 같은 상자를 여러 번 옮겨 담고, **트럭 두 대 중 한 대는 세워둔 채였다.**

그 "제3의 무언가"가 정확히 무엇이었는지가 §4(최적화 원시)와 §9(여정)의 내용이다. 미리 말하면 세워둔 트럭 한 대가 제일 컸다.

2.7 그런데 루프라인이 틀렸다 — 실물 비용은 바이트가 아니다

§2.6의 계산은 교과서적으로 옳지만, **이 하드웨어에서는 틀린 답을 준다.** 165번의 실험 끝에 알아낸 진짜 비용 모델을 여기 적는다. 이걸 모르면 §9의 후반부가 이해되지 않는다.

2.7.1 증상 — 정적 모델이 qkv에서만 계속 틀렸다

오랫동안 qkv 하나만 최적화에 반응하지 않았다. 증상이 이랬다.

- 정적 스케줄은 x 복제 로드를 **18.4k cycle**로 썼는데, 실물에서는 **~45k**였다.
- 실측/makespan 비가 **qkv만 2.4~2.7**이고 다른 두 커널은 2.05~2.2였다.
- **바이트를 절반으로 줄여도(V138)** 실측이 안 움직였다.
- HBM 사본을 늘려도(V52) 오히려 **+10k** 느려졌다.

바이트를 줄였는데 시간이 안 줄면, **시간을 먹는 것이 바이트가 아니라는 뜻이다.**

2.7.2 원인 — 커널은 ARM 코어가 돌리는 프로그램이다

런타임 소스를 읽고 나서야 답이 나왔다. **PE 안의 커널은 ARM 코어에서 도는 프로그램이고, DMA 디스크립터를 런타임에 직렬로 만들어 발행한다.**

그러니 비용의 축이 둘이다.

무엇	비용
옮기는 바이트	대역폭 (교과서적 루프라인)
발행하는 디스크립터 개수	ARM 코어의 직렬 작업 ← 이쪽이 지배적일 수 있다

문제의 x 복제 로드는 **512개 슬라이스가 같은 7.7 KB를 각자 읽는** 형태였다. 바이트로는 7.7 KB지만 디스크립터는 **512개**다. ARM 코어가 그걸 하나씩 만들어 내보내는 동안 DMA 엔진은 놀고 있었다.

이래서 §2.6의 "트럭 비유"를 고쳐야 한다. 도로가 좁은 게 아니라(대역폭), **배차 담당자가 운송장을 손으로 한 장씩 쓰고 있었다.** 짐을 줄여도 운송장 수가 그대로면 아무 소용이 없다.

2.7.3 해법 — 한 번 읽고 칩 안에서 뿌린다

고치는 방법은 자연스럽다. HBM에서 조금만 읽고, 나머지는 칩 내부 네트워크로 복제한다.

전: 512 슬라이스 × 각자 HBM 읽기 = 디스크립터 512개

후: 클러스터당 8부만 HBM 읽기 (16 디스크립터) + ring-32 switch broadcast = 디스크립터 16개 (-496)

이게 v158 이고, qkv가 151k → 108k (-28%) 로 떨어졌다. 그리고 이 하나로 프로젝트 전체 점수가 5.03 → 5.6 대로 올랐다.

§4 원시 4(경유)에서 "온칩 브로드캐스트보다 HBM 왕복이 빠르다"고 했던 것을 기억하는가? 그 결론이 뒤집힌 게 아니라, 조건이 밝혀진 것이다. ring switch가 비싼 게 아니라 어떤 switch가 비싼지가 문제였고, HBM 왕복이 싼 게 아니라 디스크립터 수가 적을 때만 싼다.

함정 하나 (실측으로 배움). 첫 구현(V154/V155)은 cluster 1이 전부 쓰레기였다. 더미 클러스터 측 위의 pass는 cluster 0에서만 돈다. 실제 측(q_s / 2048) 위에서 로드·switch하고 끝에서 Replicated 로 reshape 해야 두 클러스터가 다 채워진다.

2.7.4 세 커널이 같은 값에 모였다 — 그리고 우리는 그걸 오독했다

세 커널을 전부 §2.7.3의 원리로 정리하고 나니 셋 다 같은 값에 붙었다.

커널	바이트	V0	V165	실효 전송률
qkv	30.0 MB	126 B/cyc	105,544	298 B/cycle
attn_out	15.0 MB	39 B/cyc	53,374	295 B/cycle
ffn	94.9 MB	27 B/cyc	349,160	285 B/cycle

서로 다른 코드가 서로 다른 최적화를 거쳐 5% 안에 모였으니 공통의 물리적 한계에 도달했다 — 당시 이 문서가 내린 결론이고, 그럴듯했다. 실제로 ffn에서 계산을 지워봐도(V170) 시간이 안 줄었으니 근거도 있었다.

그런데 틀렸다.

2.7.5 DMA 비용 곡선을 직접 재보니 — 벽이 아니라 잘못된 지점이었다

"한계"라고 부르기 전에 재봤어야 했다. V197이 그걸 했다. 고정된 3.93 MB를 런 길이만 바꿔가며(7,680 / 3,840 / ... / 120 B) 추가로 읽어 비용 = f(런 길이) 곡선을 실측했다.

런 길이	증분	실효 속도
128 B	+49,341	319 B/cycle
256 B	+25,430	618 B/cycle ← 최적
512 B	+26,430	595 B/cycle
1024 B	+28,222	557 B/cycle
2048 B	+28,762	547 B/cycle

세 가지가 한꺼번에 확정됐다.

1. **256 B 미만은 정확히 2× 벌점.** 128 B가 256 B의 **1.94배**다. 문서에 적힌 "256 B 정렬 위반 시 읽기 2×"가 실물로 확인됐다.
2. **최적은 256 B이고 더 길면 완만히 나빠진다.** 256 → 2048에서 13% 악화. "길수록 좋다"는 통념이 틀렸다.
3. **최고 실효 속도는 618 B/cycle = 스펙 1,500 B/cycle의 41%.** 우리가 "벽"이라 부른 ~290은 그 절반도 안 됐다.

오독의 교훈. 서로 다른 것들이 같은 값에 모이면 물리적 한계처럼 보인다. 하지만 **공통의 실수도 똑같이 그렇게 보인다.** 세 커널이 전부 같은 "런 길이가 나쁜 상태"에 있었을 뿐이었다. 수렴은 한계의 증거가 아니다.

2.7.6 그런데 고칠 수 있는 커널이 하나뿐이다

곡선이 최적점을 알려줬어도 **거기로 옮기려면 분할 축이 2의 거듭제곱이어야 한다.** 256 B로 딱 떨어져야 하기 때문이다.

커널	분할 축	2의 거듭제곱?
attn_out	$Q_s = 4096$	✓
qkv	$H = 3840$	✗
ffn	$H = 3840$	✗

attn_out만 가능했고, 실제로 통했다 — 8청크(512 B 런) → 16청크(256 B 런)로 바꾸니 **warm -6.5% / cold -9.9%, 4/4 잡 승(V198).** 분산도 절반 이하로 줄었다.

qkv와 ffn은 H=3840에 막혀 있다. $3840 = 2^8 \times 15$ 라 256 B 경계에 맞출 수가 없다. **이게 이 프로젝트에 남은 가장 큰 미해결 축이다**(§10).

2.7.7 곡선이 과거의 해석도 고쳤다

V197의 곡선은 앞선 결론 하나를 뒤집었다. V181의 ffn -31k를 이 문서는 "가중치 스케일을 연속 1세그먼트로 만들어서"라고 설명했었다. 그런데 **57,600 B 런은 곡선상 최악 구간이다.** 런 길이로는 오히려 손해였다는 뜻이고, 실제 이득은 **inter-slice reduce 제거**에서 왔다.

같은 이유로 후속 계획(V200/V202의 ffn 청크 재분할)도 전제가 무너져 취소됐다 — ffn down은 weight 960 B-scale 120 B로 **8청크가 이미 최적**이었다(4청크는 weight를 1920 B로 늘려 손해, 16청크는 scale을 60 B로 만들

`Cluster = m![1 # 2]` 가 "1개 클러스터 분량의 일을 2개 클러스터 자리에 복제" = 절반이 노는 상태였던 이유다. `v14` 는 이걸 `m![Qs / 2048]` (= 2, 진짜 논리적 2개)로 바꾼다.

③ = — 부분만 잘라 본다

`m![H = 480 # 3840]` → 전체 3840 중 480칸짜리 조각

`tile()` 로 큰 텐서의 일부만 볼 때 쓴다.

3.2 커널 체인 해부 — `begin` 부터 `commit` 까지

`furiosa-opt` 커널은 **파이프라인 선언문**의 나열이다. 하나하나가 `ctx.<엔진>.begin(입력)` 으로 시작해서 `.commit()` 으로 끝나고, 그 사이의 각 단계가 실제 하드웨어 명령이 된다.

```
let contraction = ctx

  .main                                // ㉑ 어느 일꾼에게 시킬지
  .begin(weight_f8.view())             // ㉒ 입력: DM에 있는 f8 가중치
  .fetch::<OUTER, INNER>()             // ㉓ DM에서 어떤 모양으로 꺼낼지
  .fetch_table_lookup::<f8e4m3>()      // ㉔ (선택) 꺼내면서 타입 변환
  .collect::<OUTER2, INNER2>()         // ㉕ 레지스터에 어떤 모양으로 모을지
  .contract_outer::<OUTER, INNER, __, __>(&x_trf) // ㉖ 1
  .contract_packet::<R1>()              // ㉗ | contraction 4단
  .contract_time::<R2>()                // ㉘ | (§3.3)
  .contract_lane::<R3, LANES>(LaneMode::...) // ㉙ J
  .cast::<bf16, m![1 # 16]>()           // ㉚ (선택) 결과 타입 변환
  .transpose::<...>()                  // ㉛ (선택) 축 순서 정리
  .commit_trim::<...>()                // ㉜ 패딩 잘라내기
  .commit();                           // ㉝ DM에 쓰기
```

세 자리만 이해하면 된다.

fetch — 읽으면서 공짜로 할 수 있는 일. DM에서 데이터를 꺼내는 단계인데, 여기에 **타입 변환을 엮을 수 있다** (㉔). `f4` 가중치를 꺼내면서 `f8`로 퍼는 게 대표적이다. 이걸 따로 하면 "전체를 읽어 변환해서 DM에 쓰고, 다시 읽는" 패스가 하나 더 생긴다. 그 융합이 실제 최적화였다(\$4 원시 1).

제약 (실측으로 확인). LUT는 **연달아 못 건다**. `f4 → f8` (paired)과 `f8 → bf16` (non-paired)을 한 `fetch`에 이어 붙일 수 없어서, NVFP4 가중치를 `bf16`으로 만들려면 DM 사본을 반드시 한 번 거쳐야 한다. 이 제약이 "그럼 `bf16`으로 안 만들고 `f8`인 채로 곱하자"는 결론(\$4 원시 10)으로 이어졌다.

commit — 곧 "DM에 쓰기"다. 그래서 **체인 개수 = DM 왕복 횟수**이고, 체인을 합치는 게 곧 최적화다.

`commit_view()` 를 쓰면 큰 텐서의 특정 타일 자리에 직접 쓸 수 있어서 중간 복사가 사라진다.

나머지 축 인자들은 전부 "모양"이다. `m![...]` 안의 값이 바뀌면 계산 결과는 그대로인데 스케줄이 달라진다. §3.3 이 그 이야기다.

3.3 contraction 4단 파이프라인 — 이 문서의 핵심

§2.2에서 contraction은 "공통 축에서 곱하고 그 축을 따라 더해 없애는 것"이라고 했다. 여기서는 **RNGD가 그걸 실제로 어떻게 하는지, 그리고 왜 같은 수식이 열 배 차이 나는지를 본다.**

3.3.1 왜 4단인가 — 한 번에 3,840개를 못 더한다

Q 투영의 한 출력은 이렇게 생겼다.

$$q_o = \sum_{h=0}^{3839} W_{oh} x_h$$

3,840개 항을 한 사이클에 더하는 하드웨어는 없다. 곱셈기도 유한하고, 덧셈 트리도 유한하다. 그래서 이 합을 세 겹으로 쪼개 나눠 처리한다. 덧셈의 결합법칙이라 결과는 같다.

$$\sum_{h=0}^{3839} = \underbrace{\sum_{\text{lane}}}_{\text{공간}} \underbrace{\sum_{\text{time}}}_{\text{시간}} \underbrace{\sum_{\text{packet}}}_{\text{공간}}$$

각 겹이 하드웨어의 서로 다른 부분에 대응한다.

겹	성격	무엇인가
Packet	공간	한 번에 스트림되는 최소 묶음 (32/64 B). 곱셈기 배열이 동시에 처리
Time	시간	그냥 루프. 누산기에 계속 더한다
Lane	공간	TRF의 병렬 차선. 마지막에 접는다

그리고 이 세 겹을 조작하는 게 네 개의 호출이다.

contract_outer

↓

contract_packet

↓

contract_time

↓

contract_lane

두 텐서를 짝짓고 어느 축을 접을지 선언 (자연 실행 - 아직 계산 안 함)

Packet Reducer: 묶음 안에서 더한다

Time Reducer: 루프 돌며 누산한다

Lane Folder: 차선을 접어 최종 출력을 만든다

읽을 때 헷갈리는 지점 하나. 각 단계의 타입 인자는 "무엇을 줄일지"가 아니라 "줄이고 나서 무엇이 남는 지"다. 그래서 `contract_packet::<m![1]>()` 은 "1만 남긴다" = 묶음을 통째로 다 더한다는 뜻이다.

3.3.2 예제 A — Q 투영 (전부 다 더하는 경우)

`src/device/sliding/projection.rs` 의 실제 코드다.

```

let contraction: DmTensor<bf16, Chip, QueryClusters, QueryRows, m![Qs % 8]> = ctx
    .main
    .begin(weight_f8.view())
    .fetch::<m![Qs % 8, H / 64, Dummy2], m![H % 64]>()
    .collect::<m![Qs % 8, H / 64, Dummy2, H / 32 % 2], m![H % 32]>()
    .contract_outer::<m![Qs % 8, H / 64, Dummy2], m![H % 64], _, _, _>(&x_trf)
    .contract_packet::<m![1]>()
    .contract_time::<m![Qs % 8]>()
    .contract_lane::<m![Qs % 8], m![1 # 8]>(LaneMode::Interleaved)
    .cast::<bf16, m![1 # 16]>()
    .transpose::<m![Qs / 4 % 2], m![Qs % 4 # 16]>()
    .commit_trim::<m![Qs % 4]>()
    .commit();

```

이 슬라이스가 맡은 일. Q 가중치 $[4096 \times 3840]$ 중 8행을 갖고 있다(4096행을 2 클러스터 $\times$ 256 슬라이스 = 512로 나눔). 그 8행 각각을 $x(3,840)$ 와 내적해서 8개의 출력을 만든다.

모양을 따라가 보자. $H = 3840$, $Qs \% 8 = 8$, $H / 64 = 60$, $Dummy2 = 2$.

단계	인자	무슨 일이 일어나나
<code>contract_outer</code>	outer <code>m![8, 60, 2]</code> , inner <code>m![H % 64]</code>	스트림 스텝이 $8 \times 60 \times 2 = 960$ 개. 스텝마다 64개 원소를 소비
<code>contract_packet::<m![1]></code>	남는 것: 1	64개를 전부 더해 스텝당 값 1개
<code>contract_time::<m![8]></code>	남는 것: 8행	$60 \times 2 = 120$ 스텝을 누산. 행마다 하나씩, 8개 남음
<code>contract_lane::<m![8], m![1 # 8]></code>	남는 것: 8	차선을 접어 최종 8개 출력

검산: 8행 $\times$ 60청크 $\times$ 64원소 = 3,840 $\checkmark$. 슬라이스당 MAC 수는 $8 \times 3,840 \times 2 = 61,440$.

Dummy2 가 왜 거기 있는지가 이 예제의 진짜 요점이다. §4 원시 10에서 설명할 hi/lo 분해 — bf16 활성값 x 를 f8 두 조각으로 쪼개는 기법 — 이 contraction의 time 축 하나로 구현되어 있다.

$$\sum_h W_{oh} x_h = \sum_h W_{oh} hi_h + \sum_h W_{oh} lo_h$$

Dummy2 를 outer에 끼워 넣으면 가중치 패킷이 두 번 흐르고, Time Reducer가 같은 누산기에 두 내적을 알아서 더한다. 별도 패스도, 별도 덧셈도 없다. 정밀도를 두 배로 쓰면서 f8 경로(BF16의 2배 처리량)를 타는 값이 time 축 하나인 셈이다.

이게 왜 필요한지는 실측이 증명했다. hi 조각 하나만 쓰는 변형(V138)을 재봤더니 q 11.69% / k 27.53%의 최대 오차로 정확도 FAIL이었다. f8e4m3의 가수 3비트는 3,840항 내적에서 오차가 충분히 상쇄되지 않는다. lo 조각은 장식이 아니다.

3.3.3 예제 B — FFN up (일부러 덜 더하는 경우)

같은 4단인데 `contract_packet` 인자만 다르다. `src/device/shared/mlp.rs` :

```
.fetch::[L % 60 = $rows, H / 64 % 30, Dummy2], m![H % 64]>()
.fetch_table_lookup::[L % 60 = $rows, H / 64 % 30, Dummy2, H / 32 % 2], m![H % 32]>()
.contract_outer::[L % 60 = $rows, H / 64 % 30, Dummy2], m![H % 64], _, _, _>(x_trf)
.contract_packet::[H / 16 % 4]>() // ← 10이 아니라 4를 남긴다
.contract_time::[L % 60 = $rows, H / 64 % 30]>()
.contract_lane::[L % 60 = $rows, H / 64 % 30], m![H / 16 % 4 # 8]>(LaneMode::Sequential)
```

`contract_packet::[H / 16 % 4]>()` — 64개를 1개로 접지 않고 4개로 접는다. 즉 16개짜리 블록마다 부분합 하나를 남긴다.

왜 이 이상한 짓을 하나? **FFN 가중치가 NVFP4**라서다. 4비트 값 16개마다 스케일이 하나씩 붙는다.

$$w_{\text{실제}} = w_{4\text{bit}} \times s_{\text{블록}} \times s_{\text{전역}}$$

여기서 갈림길이 생긴다.

방식	스케일을 언제 곱하나	곱셈 횟수
순진한 방식	가중치 원소마다 (contraction 전)	$15,360 \times 3,840 = 59,000,000$
부분합 방식	16개 블록 부분합마다 (contraction 후)	$15,360 \times 240 = 3,686,400$

$$\sum_b s_b \left(\sum_{j \in b} w_j x_j \right) = \sum_j (w_j s_b) x_j$$

수학적으로 완전히 같은데 벡터 엔진 작업이 16배 줄어든다. 달라지는 건 f32 합산 순서뿐이다. `contract_packet` 인자를 1 대신 4로 쓴 것, 그게 전부다.

부분합은 `commit_view` 로 버퍼에 쌓아뒀다가, 다음 패스(pass B)에서 블록 스케일을 곱하고 `vector_inter_slice_reduce(Add)` 로 슬라이스를 가로질러 마저 더한다.

같은 수식, 같은 4단 호출, 인자 하나 차이로 벡터 부하가 16배. 이게 §3.3의 결론이다.

3.3.4 LaneMode — 차선을 어떻게 접나

저장소에서 쓰이는 두 모드의 용법은 이렇게 갈린다.

모드	언제 쓰나	예
Interleaved	차선마다 독립적인 출력 행이 나올 때	예제 A (8행 = 8개 출력)
Sequential	차선이 순서 있는 부분합을 실어 나르고 뒤에서 마저 줄일 때	예제 B (16열 블록 부분합)

3.3.5 나머지 축이 슬라이스를 넘어갈 때

접을 축이 한 슬라이스 안에 다 들어가지 않으면(FFN down의 `L = 15,360` 처럼) 슬라이스끼리 나눠 갖고, contraction이 끝난 뒤 **슬라이스를 가로질러** 더해야 한다.

```
.vector_inter_slice_reduce::<DownRows, m![H % 60 = $rows]>(InterSliceReduceOpF32::Add)
```

이건 contraction 파이프라인이 아니라 **벡터 엔진** 쪽 연산이다. 그래서 "슬라이스를 몇 개 쓸까"는 공짜 선택이 아니다 — 슬라이스를 늘리면 각자 일은 줄지만 **마지막에 합칠 것이 늘어난다**(\$4 원시 2의 트레이드오프).

3.3.6 그래서 튜닝할 수 있는 자유도가 뭔가

같은 수식을 놓고 우리가 고르는 것들이다.

자유도	어디서 정하나	실제 사례
접는 축을 몇 겹으로 나눌지	<code>contract_outer</code> 의 inner 크기 (32/64)	전 커널
어디까지 접고 어디부터 남길지	<code>contract_packet</code> 인자	V29 (1 → 4, 벡터 부하 1/16)
몇 행씩 묶어 한 패스로 처리할지	타일 높이	V12 (4→12), V37 (60행 한 패스), V82 (pass B 12행 × 5타일)
몇 슬라이스에 펼칠지	슬라이스 매핑 타입	V2 (32→256), V14 (클러스터 2배)
어떤 타입으로 곱할지	피연산자 타입 + 분해	V31/V32 (f8 × f8)

한계도 실측으로 알게 됐다. pass B의 타일 높이는 **16이 상한**이다 — 스케일 VRF가 16 × 120 f32까지만 들어간다. V82는 그 안에서 12행 × 5타일이 16/16/16/12보다 낫다는 걸 찾은 것이고, 이득은 ffn -1.5%였다.

4. 최적화 원시 카탈로그

이 프로젝트에서 실제로 통한(또는 안 통한) 레버들이다. 각각 "무엇을 / 언제 / 실제 사례"로 정리했다. §9의 실험 목록이 전부 이 아홉 개의 조합이다.

원시 1. 융합 (fuse) — 두 패스를 한 체인으로

무엇을. 따로 하던 두 단계를 하나의 `begin()...commit()` 체인에 합친다.

왜 되나. 체인이 끝나면 결과가 DM에 써지고, 다음 체인이 그걸 다시 읽는다. 합치면 쓰기 한 번 + 읽기 한 번이 통째로 사라진다.


```
// 전: 두 체인 = DM에 30 MB 쓰고 다시 읽음
let widened = ctx.main.begin(weight_f8).fetch_table_lookup::<bf16>().commit();
let result = ctx.main.begin(widened).contract_outer(&x_trf)...commit();

// 후: 한 체인 = fetch 단계에서 변환하며 바로 접음
let result = ctx.main.begin(weight_f8)
    .fetch::<...>()
    .fetch_table_lookup::<bf16>() // ← 여기로 흡수
    .contract_outer(&x_trf)...commit();
```

실제 사례. `V10_attn_weight_tiles_fused_lut` — f8→bf16 룩업을 qkv/attn_out의 contraction 체인에 융합. attn_out 53,848, qkv 93,127로 내려갔다.

한계. 아무거나 합쳐지지 않는다. 하드웨어 파이프라인 단계 순서를 지켜야 한다(fetch 단계 변환은 되지만, contraction 뒤 벡터 연산 뒤 또 contraction은 안 된다).

원시 2. 재배치 (relayout) — 일을 몇 조각으로 나눌지 바꾸기

무엇을. 같은 데이터를 다른 슬라이스 분할로 놓는다.

왜 되나. 512개 슬라이스가 있는데 32개만 쓰고 있으면 16배 손해다. 반대로 너무 잘게 쪼개면 조각당 고정 비용 (스케일 로드, 커밋)이 조각 수만큼 붙는다. **중간 어딘가에 최적점이 있다.**

```
// 전: 32 슬라이스 × 120행
type HiddenRows = m![H / 120, 1 # 8];

// 후: 256 슬라이스 × 15행 + 슬라이스 간 합산
type HiddenRows = m![H / 60, ...];
```

실제 사례. - `V2_attnout_rows_over_256_slices` — O 투영을 32 → 256 슬라이스로. **194,020 → 106,461 (45% 감소)**. 이 프로젝트 최대 단일 개선. - `V12_ffn_rows_per_pass_12` — FFN 패스당 4행 → 12행. 패스 수가 1/3로 줄어 고정 비용 감소. - `V14_two_clusters` — 클러스터 1개 → 2개. 512 슬라이스 전부 사용.

한계. 슬라이스를 늘리면 각자 담당이 작아져서 **슬라이스 간 합산(inter-slice reduce)**이 필요해진다. 그 비용이 이득을 넘으면 역효과다.

원시 3. 엔진 이동 (offload) — 놓고 있는 일꾼에게 넘기기

무엇을. `ctx.main` 이 하던 일을 `ctx.tdma` 나 `ctx.sub` 로 옮긴다.

왜 되나. §2.5 규칙 1 — 같은 일꾼의 일은 직렬이다. 96% 바쁜 일꾼에게서 일을 덜어내면 그만큼 총 시간이 준다.

실제 사례. `v0` 에서 `MainContext` 점유율이 96%였고, 최장 노드가 `layout.rs` 의 브로드캐스트(약 62,000 cycle)였다. 이걸 옮기는 게 V7의 출발점이었다.

한계. 옮긴 곳이 새 병목이 될 수 있다. 옮기고 나서 지배 컨텍스트가 뭘로 바뀌었는지 반드시 다시 본다.

원시 4. 경유 (staging) — 빠른 우회로 찾기

무엇을. A에서 B로 직접 가는 대신, C를 거쳐 간다.

왜 되나. 직관에 반하는데, 온칩 데이터 이동이 HBM 왕복보다 느릴 수 있다. 슬라이스 512개에 값을 뿌리는 switch 는 링 구조로 256홉을 도는 연산이라 비싸다. 반면 HBM에서 DM으로 읽어오는 DMA는 "브로드캐스트 읽기"를 하드웨어가 원래 지원한다.

```
// 전: 온칩에서 512 슬라이스에 뿌리기 = 54,000~62,000 cycle
let x = layout::broadcast_hidden(ctx, &x);

// 후: HBM에 7.5 KB 썼다가 복제 로드 = HBM DMA 속도
let mut x_hbm: HbmTensor<bf16, Chip, m![H]> = HbmTensor::new();
x.view().to_hbm_view(&mut ctx.tdma, x_hbm.view_mut());
let x = x_hbm.to_dm(&mut ctx.tdma); // 512 슬라이스에 복제되어 로드됨
```

실제 사례 (실측으로 확인됨). - `v7_qkv_x_replicate_via_hbm` — 위 코드 그대로. 실측 기하평균 $1.000\times \rightarrow 2.430\times$, 3/3 PASS. 이 프로젝트 최초의 검증된 승리다. - `v9_ffn_x_via_hbm`, `v13_ffn_dma_trims` — 같은 수법을 FFN에 적용.

그리고 여기서 이 프로젝트 최대의 사고가 났다.

`v15_x_replicate_hbm_copies` 는 한 단계 더 나가려 했다. V7의 HBM 경유마저도 512개 슬라이스가 같은 7.5 KB 주소를 동시에 읽는 패턴이라 420 B/cycle밖에 안 나왔으니, 같은 벡터를 HBM에 8부 써두고 슬라이스마다 다른 사본을 읽게 하자는 것이었다. makespan이 실제로 줄었다(qkv 73,445 $\rightarrow$ 60,412).

그런데 사본이 한 번도 만들어지지 않았다. 코드는 "목적지 텐서에 사본 축을 붙이면 DMA write가 그 축을 따라 복제된다"고 가정했는데, 복제되지 않는다. 사본 1개만 채워지고 나머지 7부는 초기화되지 않은 HBM이었다. 슬라이스 8개 중 7개가 쓰레기를 x로 읽고 있었던 것이다.

makespan은 이걸 못 잡는다 — 정적 스케줄은 "얼마나 걸리는가"에 답하지 "맞는가"에 답하지 않는다. V15부터 V41까지 26세대가 이 위에 쌓였고, 첫 실측에서 전부 정확도 FAIL로 날아갔다(\$9.2).

교훈 셋. 1. "온칩이 무조건 빠르다"는 통념은 틀렸다 — V7이 반례다. 2. 중복 저장이 정당일 수 있다는 아이디어 자체는 살아 있다. 다만 사본을 진짜로 만들려면 사본마다 `to_hbm_view` 를 명시적으로 불러야 하고, 그러면 store 하나당 Core 디스크립터 명령이 3개씩 붙는다. Core는 in-order 발행이라 그게 임계경로가 된다 — 사본 수 1/2/4/8/16을 전수 측정한 결과 어디에서도 사본 없는 쪽을 못 이겼다(V49, 기각). 3. 성능이 좋아졌다는 신호를 정확도 확인 없이 믿으면 안 된다. 특히 "공짜로 늘어난 것"처럼 보일 때.

원시 5. 청킹 (chunking) — 필요한 만큼만 풀기

무엇을. 전체를 한꺼번에 처리하는 대신 조각내서, 각 조각이 필요할 때만 준비한다.

왜 되나. 압축된 가중치를 전부 풀어놓으면 DM이 넘치고, 넘치면 DM↔DM 재배포 DMA가 생긴다. 조각내면 그 재배포가 통째로 사라진다.

실제 사례. `V1_ffn_down_chunked_dequant` — down 투영에서 슬라이스별로 `L/8` 청크만 역양자화. **1,693,200 → 609,223 (64% 감소)**. FFN 최대 개선.

한계. 조각마다 고정 비용이 붙는다. 원시 2와 정확히 같은 트레이드오프다.

원시 6. 오버랩 (overlap) — 독립적인 일을 겹치기

무엇을. 서로 의존하지 않는 두 작업을 다른 일꾼에게 나눠 동시에 돌린다.

왜 되나. 점수가 되는 cycle은 **모든 작업 구간의 합집합**이다(§8.2). 겹치면 그만큼 줄어든다.

```
전: [up 가중치 로드][up 계산][gate 가중치 로드][gate 계산]
후: [up 가중치 로드][up 계산]
    [gate 가중치 로드][gate 계산]    ← DMA가 앞당겨짐
```

실제 사례. `V6_ffn_upgate_overlap` — up과 gate는 완전히 독립인데 순차 실행되고 있었다. 609,223 → 412,304.

한계. MainContext끼리는 못 겹친다(규칙 1). 한쪽을 DMA/Sub로 밀어낼 수 있을 때만 성립한다.

원시 7. 폭 확장 (widen) — 놓고 있는 하드웨어 쓰기

무엇을. 안 쓰고 있던 자원을 동원한다.

실제 사례. `V14_two_clusters` — `Cluster = m![1 # 2]` 라 두 클러스터가 같은 계산을 반복하고 있었다. 행을 실제로 나눠 512 슬라이스를 쓰자 **세 커널이 동시에 1.27× / 1.31× / 1.83×** 빨라졌고, 기하평균이 2.87× → 4.15×로 뛰었다. **이 프로젝트 단일 최대 개선이다.**

한계. 클러스터 간 데이터 합치기가 새로 필요해진다. 현재 코드는 그걸 HBM 경유(원시 4)로 푼다 — 512 슬라이스에서 16 B씩 직접 저장하면 37,000 cycle이 든다고 코드 주석에 적혀 있다.

교훈. 미세 튜닝을 열 번 하기 전에 **"자원을 다 쓰고 있는가"부터 확인해야 한다**. V1~V13이 13번의 실험으로 2.87×를 만드는 동안, 놓고 있던 절반을 켜는 한 번이 1.45×를 더 얻었다.

원시 8. 정밀도 예산 (precision budget) — 오차 여유를 성능으로 바꾸기

무엇을. 중간 계산의 f32 왕복을 bf16으로 줄인다.

왜 되나. `bf16 → f32 → 연산 → bf16` 왕복은 변환 비용 + 메모리 2배다. 정확도 여유가 있으면 생략할 수 있다.

커널마다 여유가 다르다.

커널	atol	여유
sliding_attention_output	0.05	가장 넓음
sliding_project_qkv	0.04	넓음
decoder_feedforward	0.01	거의 없음

실제 사례. 처음엔 "맨 마지막에 쓸 레버"로 미뤄졌는데, 결과적으로 후반부(V29~V35)의 주력이 됐다. 다만 쓰인 방식이 예상과 달랐다 — **정밀도를 낮춰서 얻은 게 아니라, 수학적으로 동일한 재배치를 하기 위한 여유로 썼다.**

- v29 — FFN 블록 스케일을 가중치가 아니라 **부분합에 적용**. $\sum_b s_b \sum_{j \in b} w_j x_j = \sum_j (w_j s_b) x_j$ 로 수학적으로 같고, 달라지는 건 f32 합산 순서뿐이다. 벡터 엔진 부하 121k → 45k 예상.
- v33 — 어텐션 출력을 상수 16으로 스케일. 어텐션 출력은 $\sum_j p_j v_j$ 이고 $\sum p_j = 1, p_j \geq 0$ 이라 값의 범위가 미리 묶여 있다. **동적으로 계산하던 스케일을 상수로 바꿔서** 계산 체인 전체를 지웠다.

주의. 정확도는 hard gate다. 틀리면 아무리 빨라도 **0점**이다(\$8.3). 위 변경들은 전부 **makespan**에서만 검증됐고 **실측 정확도는 미확인**이다. RESULTS.md에도 "실측 검증 필요"로 적혀 있다.

원시 10. 분해 (split) — 느린 타입을 빠른 타입 둘로 쪼개기

무엇을. bf16 값 하나를 f8 두 조각(hi, lo)으로 나눠서, 느린 bf16 경로 대신 빠른 f8 경로를 두 번 탄다.

왜 되나. §2.1에서 봤듯 RNGD는 FP8이 BF16의 2배다. 그런데 $f8 \times f8$ contraction을 하려면 **양쪽 다 f8**이어야 하는데, 활성값은 bf16이다. 그래서 활성값을 $x \approx (hi + lo) \cdot 2^{-k}$ 로 쪼갠 뒤 가중치 패킷을 시간축으로 두 번 흘린다. 정밀도는 두 조각의 합으로 복원된다.

$$\sum_j w_j x_j = 2^{-k} \left(\sum_j w_j hi_j + \sum_j w_j lo_j \right)$$

결정적인 부수 효과. f8로 직접 곱하면 **f8→bf16 LUT 패스가 통째로 사라진다**. 원시 1이 LUT를 contraction에 융합했다면, 이건 LUT 자체를 없앤다.

실제 사례. v31 (qkv) — LUT 패스 5,063과 그에 딸린 DMA가 사라지고 K/V contraction이 2,663 → 1,225.

v32 (attn_out) — 같은 수법, 30,037 → 29,318. v29 (FFN)에서 먼저 쓰인 기법이다.

공짜로 얻은 것. 결과가 2^k 배로 커진 채 나오는데, **뒤따르는 RMSNorm이 스케일 불변**이라($\text{rms}(s \cdot q) = s \cdot \text{rms}(q)$) 되돌릴 필요가 없다. eps 항만 s^2 분의 1로 작아지는데 상대 오차 $1e-6$ 이하다. 정규화가 뒤에 있는 구조라서 성립하는 트릭이다.

한계. hi/lo 분해는 $|x| > \max |x|/4096$ 구간에서만 exact고, 그 아래는 $2^{-10}/s$ 수준의 절대 오차가 난다. 그리고 hi/lo를 HBM에 두 번 저장해야 해서 store가 늘어난다 — 그걸 한 번으로 줄이려던 v39는 컴파일 실패했다(\$9.4).

실측이 이 원시를 두 방향에서 확인했다. ① 조각을 하나만 쓰면(v138, hi만) q 11.69% / k 27.53% 오차로 **정확도 FAIL**이다 — f8e4m3의 가수 3비트는 3,840항 내적에서 상쇄되지 않는다. lo는 장식이 아니다. ② 그런데 두 조각을 다 써도 **qkv 실측은 안 줄었다**(makespan -16%인데 실측 무변화). 분해는 **정확도를 지키는 값**이지 qkv에서 속도를 버는 수단이 아니었다(\$9.4).

원시 9. 중복 제거 (dedup) — 같은 일 두 번 하지 않기

무엇을. 이미 되어 있는 걸 다시 하는 코드를 찾아 지운다.

실제 사례(부분 성공). `shared::rmsnorm::normalize`의 마지막 단계가 이미 256 슬라이스에 복제해서 돌려주는데, `ops.rs`가 곧바로 `broadcast_hidden`으로 또 복제하고 있었다. 다만 **타입만 재라벨링하는 것으로는 안 됐고**(물리 배치가 기대와 달랐다), 결국 원시 4(HBM 경유)로 우회해서 풀었다.

교훈. 코드를 보고 "중복 같다"고 판단해도 **하드웨어 배치는 다를 수 있다**. 스케줄 dump로 확인하는 게 먼저다.

원시 12. 브로드캐스트 (broadcast) — 디스크립터를 줄이는 유일한 손잡이

무엇을. HBM에서 조금만 읽고, 나머지 슬라이스는 **칩 내부 switch 네트워크**로 채운다.

왜 되나. §2.7의 비용 모델이 그대로 근거다. 512개 슬라이스가 같은 영역을 각자 읽으면 **디스크립터 512개**이고, 그걸 ARM 코어가 직렬로 만든다. 한 번 읽어 ring switch로 뿌리면 디스크립터가 16개로 준다.

전: 512 슬라이스 각자 HBM 읽기 디스크립터 512
후: 클러스터당 8부 읽기 + ring-32 switch broadcast 디스크립터 16 (-496)

실제 사례. `v157 / v158` — qkv의 x 복제. **151k → 108k (-28%)**, 프로젝트 점수 5.03 → 5.6대. 이 프로젝트 후반부 최대의 단일 개선이다. 같은 원리를 `ffn-attn_out`의 x 조각(V159/V161)과 V181의 up/gate x에도 적용했다.

한계와 함정. - **더미 축 위에서는 cluster 0만 돈다.** 실제 클러스터 축(`qs / 2048`) 위에서 로드-switch하고 끝에서 `Replicated`로 reshape해야 한다. 첫 구현(V154/V155)이 여기서 cluster 1을 통째로 쓰레기로 만들었다. - **switch가 항상 싼 것은 아니다.** 출력을 모으는 ring-256 gather는 오히려 비쌌다(V167/V168 둘 다 기각). 패딩 레이아웃의 64-디스크립터 store가 더쌌다. - **시간축 → 슬라이스축 switch는 불가능하다.**

§4 원시 4(경유)의 "HBM 왕복이 온칩 브로드캐스트보다 빠르다"와 모순처럼 보이지만 아니다. **둘 다 디스크립터 수의 함수였을 뿐이다.** V7 시점의 온칩 브로드캐스트는 ring-256 switch였고, V158의 것은 ring-32다. 바이트가 아니라 개수를 세는 순간 둘이 같은 규칙으로 설명된다.

원시 13. 런 길이 정렬 (run alignment) — 곡선의 최적점으로 옮기기

무엇을. 한 번의 DMA가 연속으로 읽는 길이(런)를 **256 B**에 맞춘다.

왜 되나. §2.7.5의 실측 곡선이 전부다. 256 B가 최적(618 B/cycle)이고 **그 미만은 정확히 2x 벌점**, 그 이상은 완만히 나빠진다. 대부분의 커널은 이 최적점에서 벗어난 곳에 있다.

`attn_out`의 `Qs=4096` 분할:
8청크 → 런 512 B → 595 B/cycle
16청크 → 런 256 B → 618 B/cycle ← 여기로

실제 사례. v198 / v200 — attn_out의 O-weight 열 분할을 8 → 16청크로. warm -6.5% / cold -9.9%, 4/4 잡 승. 분산도 절반 이하(stddev 2,461~3,165 → 561~1,605)로 줄었다. 슬라이스당 60행 → 120행으로 바뀌 256 슬라이스를 계속 live로 유지했다.

적용 조건이 까다롭다. - 분할 축이 2의 거듭제곱이어야 한다. qs =4096은 되고 h =3840은 안 된다(= 2^8 x 15). - 행 수는 4의 배수여야 한다(transpose 제약). - reduce 축은 최내측을 유지해야 한다.

"길수록 좋다"를 버려야 한다. 큰 연속 블록이 항상 낫다는 직관이 여기서는 틀린다. V181의 57,600 B 런은 곡선상 최악 구간이었고, 그 변경의 이득은 런 길이가 아니라 다른 데서 왔다(\$2.7.7).

원시 14. 명령 병합 (command merge) — 위치가 값을 정한다

무엇을. DMA 명령 두 개를 하나로 합친다.

왜 되나 — 그리고 언제 안 되나. §2.7.2에서 디스크립터 수가 비용이라고 했으니 합치면 항상 이득일 것 같지만, 아니다. 값을 정하는 것은 개수가 아니라 그 명령이 커널 선두에서 weight 스트림을 막고 있는지다.

어디서 합쳤나	결과
ffn 선두의 스테이징 store 2개 → 1개	-11,078 (-3.5%), 6/6 잡 승
같은 병합을 중반에서 (V201)	0
qkv의 같은 store	중립(평균 +447, 노이즈)

실제 사례. v199 / v200 . 선두에서만 값이 있었다.

원시 12(브로드캐스트)와 짝을 이룬다. 둘 다 디스크립터를 줄이지만, 줄이는 위치가 임계경로 위인지가 실제 이득을 가른다.

원시 요약표

검증 상태를 같이 표시한다. ✓ = 실측 PASS 계보에 포함, ⚠ = makespan만 (V15 계보에 얹혀 있어 재검증 필요), ✗ = 실측으로 기각.

#	원시	한 줄	실제 사례	검증	효과
1	융합	두 체인을 하나로	V10 / V16, V18, V23, V36, V38	⚠	대
2	재배치	슬라이스 분할 바꾸기	V2 / V12, V19, V20, V22, V28, V37	✅ ⚠	대
3	엔진 이동	놓고 있는 일꾼에게	(V7의 동기)	✅	중
4	경유	HBM 우회가 더 빠름	V7, V9, V13 / V15, V24	✅ ❌	대
5	청킹	필요한 만큼만 풀기	V1	✅	대
6	오버랩	독립 작업 겹치기	V6, V17	⚠	중
7	폭 확장	놀던 하드웨어 쓰기	V14, V25	✅	최대
8	정밀도 예산	오차 여유로 재배치 허가받기	V29 / V33, V35	✅ ⚠	대
9	중복 제거	두 번 하지 않기	(V7로 흡수)	—	—
10	분해	bf16을 f8 둘로 쪼개 빠른 경로	V29, V31, V32 (V138 기각)	✅	정확도용
11	되돌리기	틀린 전제에 기댄 것을 빼기	V50	✅	대
12	브로드캐스트	한 번 읽고 칩 안에서 뿌려 디스크립터를 줄이기	V157, V158, V181	✅	최대
13	런 길이 정렬	DMA 런을 256 B에 맞추기 (미만은 2× 벌점)	V198, V200	✅	대 (적용 가능한 곳에서)
14	명령 병합	디스크립터 합치기 — 단 커널 선두에 서만	V199, V200	✅	중

패턴이 보인다. 큰 효과를 낸 것은 전부 "데이터를 어떤 타입으로, 어디에, 몇 조각으로 놓고, 몇 번 옮길지"를 바꾼 것이다. 곱셈 횟수는 처음부터 끝까지 한 번도 줄이지 않았는데 실측 4.97배가 나왔다. §2.6의 결론 그대로 — 메모리 바운드 워크로드에서는 배치가 전부다.

실측으로 확정된 세 번의 SOTA 갱신만 떼어보면 이렇다.

갱신	주력 원시	기하평균
V7 (V1+V2+V7)	5 청킹 · 2 재배치 · 4 경유	1.000 → 2.430×
V14	7 폭 확장	2.430 → 3.936× (+62%)
V50	11 되돌리기	3.936 → 4.927× (+25%)
V82	2 재배치 (FFN pass B 타일 12×5)	4.927 → 5.086× (공식)
V165	12 브로드캐스트 (qkv x를 ring-32로)	5.086 → 5.758× (공식, +13%)
V204	13 런 길이 정렬 + 14 명령 병합	5.758 → 5.903× (공식) / Arena 6.229

큰 갱신 셋이 전부 튜닝이 아니었다. 놓고 있던 클러스터를 컨 것(V14), 틀린 코드를 지운 것(V50), 그리고 비용 모델 자체를 잘못 알고 있었음을 발견한 것(V165)이다. 미세 튜닝 수십 번보다 그 셋이 컸다.

원시 11(되돌리기)이 카탈로그에 있는 게 이상해 보이지만, 최적화 프로젝트에서 실제로 가장 자주 필요한 동작이다. 쌓아 올린 것 중 어느 층이 썩었는지 찾아 빼는 일 말이다.

5. 대회 규칙과 점수

5.1 일정

항목	날짜
등록 마감	2026-09-15
Stage 1 (커널)	2026-09-01 ~ 09-25
결선 발표	09-30
Stage 2 (E2E)	10-01 ~ 10-25
시상 (MICRO 2026, Athens)	11-01

5.2 점수 = 세 커널 speedup의 기하평균

$$\text{점수} = \sqrt[3]{\frac{c_{qkv}^{base}}{c_{qkv}} \times \frac{c_{attn}^{base}}{c_{attn}} \times \frac{c_{ffn}^{base}}{c_{ffn}}}$$

기하평균이라는 게 전략을 규정한다.

시나리오	qkv	attn_out	ffn	점수
하나만 2배	2.00	1.00	1.00	1.260
셋 다 1.3배	1.30	1.30	1.30	1.300
하나 2배 + 하나 20% 퇴보	2.00	1.00	0.80	1.170

"한 커널 2배"보다 "세 커널 30%씩"이 낫다. 그리고 어디 하나를 퇴보시키면 다른 곳의 성과를 통째로 까먹는다.

지금 우리 상황에 대입해보면 이렇다.

	현재 (V165 공식)	어느 커널이든 2배 되면
qkv	2.37×	—
attn_out	7.58×	—
ffn	10.61×	—
기하평균	5.76×	7.25×

어느 커널을 2배 만들든 기하평균에 미치는 효과는 똑같다. 기하평균이 곱의 세제곱근이라 그렇다. 그러니 "어디가 제일 느린가"가 아니라 "어디가 제일 쉽게 2배가 되는가"로 골라야 한다.

그런데 V165 시점에서는 이 계산이 더 이상 안내가 되지 않는다. 세 커널이 전부 ~290 B/cycle 벽에 붙어서(\$7.4), 어느 하나를 2배로 만들 여지가 구조적으로 남아 있지 않다. 남은 것은 읽는 바이트를 줄이거나 더 연속적으로 읽는 것뿐이고, 그건 세 커널에 고르게 적용된다(\$9.6).

이 동조 현상 자체가 정보다. 커널마다 다른 최적화를 했는데도 결과가 같이 움직였다면, 남은 여유는 개별 커널의 코드가 아니라 세 커널이 공유하는 무언가(하드웨어 배치, 톨체인 제약, 혹은 하한 추정 자체)에 걸려 있을 가능성이 크다. §10 열린 질문 5번이 이것을 묻는다.

5.3 고쳐도 되는 곳 / 안 되는 곳

★ 채점 반영 / ✕ 무시(baseline으로 되돌림)

범위	
src/device/ 전체	★
src/ops.rs, ops_vision.rs, ops_audio.rs 의 함수 본문만	★
src/axes.rs, src/host/, src/api/, src/bin/, src/lib.rs, tests/	✕

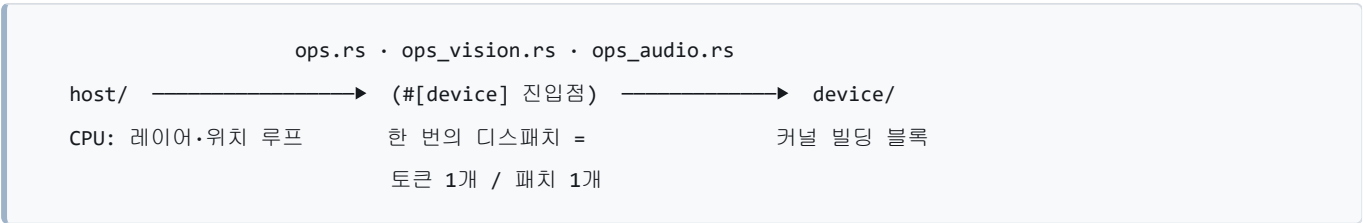
절대 규칙 넷.

- 1. #[device] 함수의 이름·인자·타입·반환형을 바꾸지 않는다. 시그니처가 평가자 계약이다.
- 2. ops*.rs 는 크레이트 루트에 그대로 둔다. 커널 이름에 module_path!() 가 들어간다.
- 3. tests/test_kernels.rs 를 고쳐서 통과시키지 않는다. 채점 서버는 자기 버전을 쓴다.
- 4. 정확도는 hard gate다. 틀리면 0점.

✕ 영역에서 빨라진 건 전부 가짜다. 설계 단계에서 배제한다.

6. 코드 지도

6.1 device / host 분리



- 커널은 작업 단위 하나만 처리한다. 배치 차원도 시퀀스 차원도 없다. 48개 레이어 루프, 토큰 위치 루프는 전부 호스트에 있다.
- ops*.rs 가 유일한 접점이다. 호스트가 부르는 #[device] 함수들.

6.2 파일별

경로	채점	내용
src/ops.rs	★	텍스트 커널 10개. Stage 1 대상 3개가 여기
src/device/layout.rs	★	Cluster / Slice / Replicated / BothClusters , 브로드캐스트 헬퍼
src/device/sliding/projection.rs	★	Q/K/V/O 투영. f8 가중치 + 채널당 bf16 스케일
src/device/sliding/rmsnorm.rs	★	헤드별 Q/K/V RMSNorm (V는 학습 가중치 없음)
src/device/sliding/rope.rs	★	$\theta=10,000$ 회전 위치 인코딩
src/device/sliding/attention.rs	★	링 KV 캐시 softmax. Stage 1 대상 아님
src/device/shared/rmsnorm.rs	★	히든 상태 RMSNorm. 세 커널 모두 사용
src/device/shared/mlp.rs	★	GeGLU FFN. NVFP4 (4비트 + 16개당 스케일 + 전역 스케일)
src/device/shared/residual.rs	★	residual add, 레이어 게이트
src/device/full/*	★	full-attention 8개 레이어. v_proj 없음, $\theta=1,000,000$
src/host/**, src/api/**, src/bin/**	×	CPU 오케스트레이션, HTTP 서버, 진입점
tests/test_kernels.rs	×	채점 기준 테스트

6.3 축(axes) 상수

축	값	의미
H	3,840	히든 크기
L	15,360	MLP 중간 크기 (= 4H)
W	262,144	어휘 크기
Ns , Gs , Ds	8, 2, 256	sliding: KV 헤드 8, 헤드당 쿼리 2, head dim 256
Qs , Ps , Ts	4,096, 2,048, 1,024	Q 폭, KV 폭, 링 캐시 길이
Gf , Df , Tf	16, 512, 512	full: 헤드 16, head dim 512, 페이지 길이

레이어 48개 = sliding 40 + full 8. **Stage 1 커널은 전부 sliding 쪽이다.**

7. 세 커널 해부

7.1 `ops::sliding_project_qkv` — 입력을 Q·K·V로 나누기

하는 일. 토큰 벡터 하나(3,840)를 정규화한 뒤 세 개의 행렬과 곱해서 Query(4,096) / Key(2,048) / Value(2,048)를 만들고, 헤드별 정규화와 위치 인코딩을 거쳐 KV 캐시에 쓴다.

```
x (3840)
├ RMSNorm                                정규화
├ 512 슬라이스에 복제                    ◀ V7: HBM 경유로 해결
├ Q 투영  Wq[4096×3840] · x              ◀ V10: LUT 융합 → V31: f8×f8로 LUT 제거
├ K 투영  Wk[2048×3840] · x
├ V 투영  Wv[2048×3840] · x
├ 헤드별 RMSNorm (Q·K는 가중치 있음, V는 없음) ◀ V36: 채널 스케일을 여기 흡수
├ RoPE (θ=10,000)                        ◀ V30: 테이블 직접 gather
└ q → 출력 / k,v → 캐시에 scatter
```

7.2 `ops::sliding_attention_output` — 어텐션 결과를 되돌리기

하는 일. 어텐션 출력(4,096)을 다시 히든 크기(3,840)로 투영하고, 정규화한 뒤 residual에 더한다.

```
attn (4096)
├ O 투영  Wo[3840×4096] · attn            ◀ V2: 32→256 슬라이스 / V32: f8×f8
├ RMSNorm                                ◀ V18: 스케일을 에필로그로 / V33: 상수 16
├ residual + x                          ◀ V11: 480×8 → 1920×2 / V16: 융합
└ residual에 쓰기
```

세 커널 중 가장 작다(가중치 15 MB). tolerance도 가장 험겁다(0.05).

7.3 `ops::decoder_feedforward` — 가장 무거운 놈

하는 일. GeGLU 방식의 2층 MLP다. 3,840 → 15,360으로 두 갈래(up, gate) 넓히고, 게이트에 GELU를 먹여 곱한 뒤, 다시 3,840으로 좁힌다.

$$\text{FFN}(x) = W_{\text{down}}(\text{GELU}(W_{\text{gate}} x) \odot W_{\text{up}} x)$$

```

residual (3840)
├─ RMSNorm
├─ 512 슬라이스에 복제                                ◀ V9: HBM 경유
├─ up 투영 Wu[15360×3840] · x                            ◀ V6: 돌을 겹침 / V12·V37: 패스 크기
├─ gate 투영 Wg[15360×3840] · x                            ◀ V13: HBM 경유 / V25: 두 클러스터
├─ GeGLU: GELU(gate) × up                                ◀ V1: 청크 역양자화 / V29: 스케일을 부분합으로
├─ down 투영 Wd[3840×15360] · g                            ◀ V16: 셋을 한 벡터 패스로 융합
├─ RMSNorm → residual add → 레이어 게이트                ◀ V38: reducing 레이어아웃에서 바로 저장
└─ residual에 쓰기

```

가중치가 NVFP4로 저장돼 있다. 4비트로 압축하고, 16개마다 f8 스케일 하나, 행렬마다 f32 전역 스케일 하나를 곱해서 원래 값을 복원한다.

$$w_{\text{실제}} = w_{4\text{bit}} \times s_{\text{블록}} \times s_{\text{전역}}$$

이 복원(역양자화)을 매번 커널 안에서 한다. 1억 7,700만 개 원소가 `f4 → f8 → f32 → x스케일 → bf16` 을 거친다. 이게 FFN이 무거운 진짜 이유고, V1의 청킹이 통한 이유다.

7.4 비용 구조 — 그리고 최적점까지 얼마나 남았나

	qkv	attn_out	ffn
가중치 (압축 상태)	30.0 MB (f8)	15.0 MB (f8)	84.4 MB (f4) + 10.5 MB (스케일)
MAC 수	31.5 M	15.7 M	176.9 M
산술 강도	2.0 FLOP/B	2.0 FLOP/B	3.56 FLOP/B
공식 baseline	250,514	404,633	3,703,473
V204 공식	101,132	56,845	317,445
speedup	2.48×	7.12×	11.67×
V200 Arena 실측	100,314	49,928	310,088
실효 전송률 (Arena)	314 B/cyc	315 B/cyc	321 B/cyc
곡선 최적점 대비	51%	51%	52%
tolerance (atol)	0.04	0.05	0.01

읽어야 할 것 하나 — 출발점이 제각각이었다. 베이스라인의 실효 전송률은 126 / 39 / 27 B/cycle이었다. 같은 "메모리 바운드 커널"인데 낭비의 양이 5배 달랐고, 그래서 ffn이 11.7배 줄어드는 동안 qkv는 2.5배에 그쳤다. qkv는 원래부터 군더더기가 적었다.

읽어야 할 것 둘 — 지금은 셋이 5% 안에 모였는데, 그게 한계는 아니다. 314 / 315 / 321 B/cycle. V165 시점에 이 수렴을 "물리적 한계"로 읽었다가 틀렸다(\$2.7.4~5). 실측 곡선의 최적점은 **618 B/cycle**이고, 우리는 그 **51%** 지점에 있다.

읽어야 할 것 셋 — 계산은 이미 전부 숨었다. ffn에서 계산을 빼봤다(V170).

무엇을 뺐나	ffn 변화
post-FF RMSNorm · residual · 레이어 게이트 (tail 전체)	-0.2k
geglu 체인 (gelu · 곱 · 스칼라 broadcast 2개)	+1.6k
gate 블록 스케일 로드 하나	+7k

계산을 지워도 ffn은 안 줄어든다. 남은 레버는 계산이 아니라 어떻게 읽는가뿐이다.

읽어야 할 것 넷 — 그런데 고칠 수 있는 커널이 하나뿐이다. 256 B 런으로 옮기려면 분할 축이 2의 거듭제곱이어야 한다. `attn_out(qs=4096)`은 되고 `qkv_ffn(h=3840)`은 안 된다(§2.7.6). `attn_out`에서는 실제로 warm -6.5% / cold -9.9%가 나왔고, 나머지 둘은 막혀 있다.

주의 — V181의 이득은 런 길이가 아니었다. 이 문서의 이전 판은 ffn -8.9%를 "연속 1세그먼트"로 설명했는데, 57,600 B 런은 곡선상 최악 구간이다. 실제 이득은 **inter-slice reduce 제거**에서 왔다(§2.7.7). 같은 오프로 세워진 후속 계획(V200/V202의 ffn 재분할)은 전제가 무너져 취소됐다.

8. 측정 체계

8.1 두 가지 숫자

지표	얻는 법	성격
makespan	<code>cargo furiosa-opt compile <kernel> --exact --dump-schedule out.json → max(lifetime.end)</code>	컴파일러의 계획표 길이. 싸고 반복 가능. 점수 아님
RNGD cycles	<code>./scripts/rngd_test.sh</code> (Arena 제출)	실측. 이것만 점수다

makespan은 스크리닝용, 실측은 판정용이다. **현재 V1~V14는 전부 makespan만 있다.** Arena 접근이 열리면 전부 다시 재야 한다.

8.2 실측 cycle의 정확한 정의

`tests/test_kernels.rs`가 `span::npu` 스패의 `begin_cycle / end_cycle`을 모아서,

```
fn window_cycles(&self) -> Option<u64> {
    let begin = spans.iter().map(|s| s.begin).min()?;
    let end   = spans.iter().map(|s| s.end).max()?;
    Some(end.saturating_sub(begin))
}
```

모든 스패의 **합집합 구간**을 그 커널의 cycle로 삼는다. 따라오는 결론 셋.

1. **디스패치 1회 전체가 측정된다.** 가중치 DMA도 포함이다. 실제 서빙이면 레이어 간 프리페치로 감출 비용도 여기서 전액 계상된다.

2. 오버랩이 그대로 점수가 된다. 합집합이므로 겹치면 준다 → 원시 6이 유효한 이유.
3. `TUC_PROFILE_LEVEL=info` 이상일 때만 수집된다. 자연 read-back이라 기본 500ms 대기 후 읽는다.

8.3 정확도 게이트

판정식은 $|기대 - 실제| \leq atol + rtol \times |기대|$, `rtol` 은 셋 다 `1e-2`.

하나라도 넘으면 그 커널은 **0점**이고, 틀린 커널의 cycle은 기록조차 하지 않는다(`FAIL(accuracy)` 로만 남긴다).

입력은 픽스처에 저장하지 않는다. `scripts/fixture_prng.py` 와 테스트의 `prng` 모듈이 **바이트 단위로 동일한** 카운터 해시로 양쪽에서 각각 합성한다. 픽스처엔 기대 출력만 있어서 약 120 KB다.

8.4 cold와 warm — 노이즈 문제를 푼 방법

§9.3에서 "qkv가 $\pm 8\%$ 흔들려 1% 개선을 판정할 수 없다"는 문제를 만났다. 그 해법이 나왔고, 이게 측정 체계에서 가장 실용적인 발견이다.

같은 프로세스 안에서 커널을 여러 번 돌리면, 첫 실행과 그 이후가 다르다.

	무엇인가	퍼짐	어디에 쓰나
cold (첫 실행)	명령 체크 스테이징이 포함된 값	$\pm 8\%$	채점 환경이 보는 값
warm (2회차 이후)	스테이징이 끝난 정상 상태	$\pm 0.7\%$	A/B 판정

여기서 운용 규칙이 나온다.

- **A/B는 warm으로 한다.** $\pm 0.7\%$ 면 1% 차이도 한 잡 안에서 갈린다.
- **보고는 cold로 한다.** 채점 서버가 보는 게 그것이다.
- **한 잡에 여러 변형을 담는다.** `tests/` 는 채점에 무시되므로, 변형 4개 × 반복 5회를 한 바이너리에 넣어 제출하면 Arena 잡 하나로 A/B가 끝난다. 제출 예산이 유한하므로 이게 중요하다.

제출 예산이 최적화의 해상도를 정한다. 공식 리더보드 제출은 하루 5회로 잡고 실측으로 확인된 것만 올린다. Arena 잡도 남발하지 않고, 잡 하나에 최대한 많은 변형을 warm 반복으로 담는다.

8.5 명령어

```
# 최초 1회
python3 scripts/generate_references.py          # → ref/fixtures.safetensors

# 스크리닝 (--exact 필수: 이름이 다른 커널의 prefix일 수 있음)
mkdir -p target/schedules
cargo furiosa-opt compile ops::sliding_project_qkv --exact \
    --dump-schedule target/schedules/V{n}_sliding_project_qkv.json
cargo furiosa-opt compile ops::sliding_attention_output --exact \
    --dump-schedule target/schedules/V{n}_sliding_attention_output.json
cargo furiosa-opt compile ops::decoder_feedforward --exact \
    --dump-schedule target/schedules/V{n}_decoder_feedforward.json

# 눈으로 보기
furiosa-schedule-viewer                        # 127.0.0.1:9254

# 판정
./scripts/rngd_test.sh                        # $RNGD_URL 필요, 사전 rngd login
```

스케줄 JSON은 스크립트로 파도 된다. `instructions[]`에 `tpe`, `contexts`, `lifetime{begin,end}`, `util{total_util}`, 소스 위치가 담긴 `description`이 있다.

9. 지금까지의 여정과 다음

9.1 일곱 번의 확정

실측으로 정확도 PASS를 받은 구성만이 점수다. 그 기준에서 이 프로젝트는 일곱 개의 점을 지나왔다. **V82·V165**는 공식 채점값이고, 앞의 넷은 우리 실측이다(분모가 조금 다르다 — §9.1 아래 주석).

시점	주제	qkv	attn_out	ffn	기하평균
V0_baseline	기준선	244,885	405,253	3,706,465	1.000×
V7 (V1+V2+V7)	명백한 낭비 제거	190,342	111,005	1,213,877	2.430×
V14	놀던 클러스터 크기	183,005	78,708	418,837	3.936×
V50	틀린 전제 되돌리기	154,324	56,216	354,403	4.927×
V82	FFN pass B 타일 12×5	155,419	52,492	349,760	5.086×
V165	qkv x를 ring-32 broadcast로	105,544	53,374	349,160	5.758×
V204	256 B 런 정렬 + 선두 명령 병합	101,132	56,845	317,445	5.903×

커널별 speedup으로 보면 서사가 한눈에 들어온다.

시점	qkv	attn_out	ffn
V7	1.29×	3.65×	3.05×
V14	1.34×	5.15×	8.85×
V50	1.59×	7.21×	10.46×
V82	1.61×	7.71×	10.59×
V165	2.37×	7.58×	10.61×
V204	2.48×	7.12×	11.67×

분모 주의. V0~V50은 우리가 켜 `v0_baseline` (244,885 / 405,253 / 3,706,465), V82~V165는 공식 `baseline`(250,514 / 404,633 / 3,703,473)이 분모다. 2% 안쪽 차이라 추세를 읽는 데는 지장이 없지만, 표의 행을 직접 빼서 쓰면 안 된다.

attn_out과 ffn은 7~10배가 됐는데 qkv만 제자리다.

9.2 첫 실측이 26세대를 지웠다

이게 이 프로젝트에서 가장 중요한 사건이라 따로 쓴다.

무슨 일이 있었나. Arena 접속 전까지 판정 기준은 makespan이었고, 그 위에서 V41까지 **5.801×**를 찍어놨었다. 첫 제출(job 15346~15352)에서 **V15부터 V41까지 전 계보가 정확도 FAIL**로 드러났다.

원인. V15가 도입한 HBM 사본 관용구가 “목적지 텐서에 사본 축을 붙이면 DMA write가 그 축을 따라 복제된 다”를 가정했는데 — **복제되지 않는다**. 사본 1개만 채워지고 나머지는 초기화되지 않은 HBM이었다. bf16 경로는 NaN, f8 경로는 220%대 오차가 났다.

두 번째 버그를 찾은 방법 (재사용할 가치가 있다). 사본을 걸어내고 다시 제출하니 v·attn_out·ffn은 PASS인데 q·k만 FAIL이었고, non-finite 위치가 index 0에서 **q=641, k=385로 옮겨갔다**. `Ds=256` 이므로

$$641 = 2 \times 256 + 129, \quad 385 = 1 \times 256 + 129$$

둘 다 같은 채널 **d=129**, 같은 kv head **n=1**이다. v만 통과한다는 사실과 합치면 남는 경로는 **RoPE뿐**이고(v는 RoPE를 안 거친다), RoPE는 Ds를 128씩 반으로 나누므로 d=129는 두 번째 half의 두 번째 원소다. 여기서 곧장 V30이 나왔다 — V30도 같은 전제로 cos/sin 테이블을 head 레이아웃에 직접 gather하고 있었다.

실패 위치의 인덱스는 커널 디버깅에서 가장 값싼 단서다. 스케줄을 뜨거나 중간값을 덤프하기 전에 인덱스 산수부터 해볼 것.

두 버그가 같은 오해 하나에서 나왔다. 서로 다른 시점에 쓴 코드인데 전제가 같았다. 정적 스케줄에서 “공짜로 늘어난 것”은 대개 늘어나지 않은 것이다.

그래서 **V50은 빼서 이겼다**. 사본을 진짜로 채우는 길(V49)을 먼저 시도했지만, store 하나당 Core 디스크립터 명령이 3개 붙고 Core는 in-order 발행이라 사본 수 1/2/4/8/16 어디에서도 사본 없는 쪽을 못 이겼다. 아이디어 자체가 틀렸으므로 결론은 “없앤다”였다.

9.3 두 번째 교훈 — 측정 자체가 시끄럽다

첫 실측이 "makespan을 믿지 마라"였다면, 그 다음에 배운 건 "실측 한 번도 믿지 마라"였다.

같은 코드를 여러 번 제출해서 재보면 이렇게 나온다.

커널	실행값	퍼짐
qkv (V50 코드, 5회)	159,405 · 153,150 · 151,686 · 149,244 · 136,576	±8%
attn_out (V50 코드, 5회)	57,892 · 56,586 · 56,522 · 55,910 · 55,446	±2%
ffn (4회)	354,825 · 353,981 · 351,454 · 346,534	±1.4%

하필 유일한 병목인 qkv가 가장 시끄럽다. 최고와 최저가 17% 차이 난다 — 같은 바이너리, 같은 픽스처인데도.

이게 실제로 사고를 낼 뻔했다. V82를 처음 제출했을 때 5.135×가 나왔는데, 다시 재니 4.929×였다. qkv가 136,576(5회 중 최저)으로 나온 운 좋은 draw였던 것이다. 평균을 내면 **V50 4.927 → V82 5.028**이고, 커널별로 보면 이렇다.

	변화	판정
ffn	-1.5% (네 실행 모두 V82가 작다)	진짜 개선 — makespan 예측(-0.26%)과 방향 일치
qkv	-6.1%	노이즈
attn_out	+1.8%	노이즈

그래서 채택 기준을 바꿨다.

1. 정확도는 단일 실행으로 판정한다. 이진값이라 노이즈가 없다.
2. 속도는 makespan이 같은 방향을 가리키고 실측이 그것과 모순되지 않을 때만 채택한다.
3. 5% 미만의 실측 차이는 단독으로는 증거가 아니다.

1% 차이를 신뢰구간 안에서 보려면 10회 이상 반복해야 하는데 제출 예산이 그걸 허용하지 않는다. 측정 예산이 최적화의 해상도를 정한다.

나중에 이 문제는 풀렸다. 노이즈의 정체가 첫 실행의 명령 청크 스테이징이었다 — 같은 프로세스에서 두 번째 실행부터는 qkv도 ±0.7%다. A/B는 warm으로 하고 보고는 cold로 하면 된다(\$8.4).

9.4 makespan은 무엇에 쓸 수 있나

두 번의 사고에서 "makespan은 쓸모없다"를 끌어내면 잘못된 학습이다. §7.4에서 봤듯 **V50의 makespan 예측(5.31×)은 실측(4.93×)과 꽤 가깝고**, attn_out·ffn은 1~2% 이내다. V82의 채택 근거도 결국 makespan이었다.

정확한 결론은 이렇다.

makespan이 답할 수 있는 것	답할 수 없는 것
이 코드가 얼마나 걸리는가 (대략)	이 코드가 맞는가
두 배치 중 어느 쪽이 빠른가 (동일 조건)	하드웨어가 이 관용구를 실제로 지원하는가
작은 변화의 방향	작은 변화의 크기 (실측 노이즈에 묻힌다)

그리고 한 커널에서는 방향조차 못 맞힌다.

9.5 qkv의 벽을 깬 방법 — 이 프로젝트의 결정적 발견

오랫동안 qkv 하나가 최적화에 반응하지 않았다. 두 번의 정면 공격이 모두 실패했다.

시도	makespan	실측	결과
v52 HBM 사본 (올바르게 구현)	개선 예측	+10k	기각
v138 x를 f8 한 조각으로 (바이트 절반)	-16%	무변화	기각 (정확도도 FAIL)

바이트를 절반으로 줄였는데 시간이 안 줄었다. 이 한 문장이 실마리였다 — 시간을 먹는 것이 바이트가 아니라는 뜻이기 때문이다.

답은 스케줄러가 아니라 런타임 소스에 있었다. PE 안의 커널은 ARM 코어에서 도는 프로그램이고, DMA 디스크립터를 런타임에 직렬로 만들어 발행한다. 문제의 x 복제 로드는 512개 슬라이스가 같은 7.7 KB를 각자 읽는 형태였다 — 바이트로는 7.7 KB지만 디스크립터가 512개였고, ARM 코어가 그걸 하나씩 만드는 동안 DMA 엔진은 놀고 있었다.

정적 스케줄이 18.4k로 세던 것이 실물에서 ~45k였던 이유, 실측/makespan 비가 qkv에서만 2.4~2.7이었던 이유, 바이트를 줄여도 안 변한 이유가 한꺼번에 설명됐다.

고친 방법은 \$4 원시 12다. 클러스터당 8부만 HBM에서 읽고(디스크립터 16개) ring-32 switch로 나머지를 채운다. 디스크립터 496개가 사라진다.

qkv 151k → 108k (-28%), 공식 점수 5.086 → 5.758 (+13%). 계산식은 한 줄도 안 바꿨다.

그리고 이 발견이 나머지 전부를 다시 보게 했다. 세그먼트 로드가 바이트당 2배 비싸다는 것(V174), ffn이 계산이 아니라 스트림에만 묶여 있다는 것(V170), 출력 gather는 오히려 비싸다는 것(V167/V168) — 전부 같은 렌즈로 설명된다. \$2.7이 이 보고서에서 가장 늦게 쓰였고 가장 중요한 절인 이유다.

9.6 메모리 대역폭 캠페인 — 벽인 줄 알았던 것을 다시 댄 이야기

V165에서 세 커널이 ~290 B/cycle에 모였을 때, 이 문서는 그것을 물리적 한계로 읽었다(\$2.7.4). 그 판단이 틀렸다는 것을 확인하는 데 Arena 잡 3개가 들었다.

V197이 한 일은 단순하다. 고정된 3.93 MB를 런 길에만 바뀌가며 추가로 읽고 시간을 잤다. 곡선이 나왔고, 최적 점은 256 B에서 618 B/cycle이었다 — 우리가 있던 지점의 두 배가 넘는다.

그리고 곡선은 세 가지를 한꺼번에 정리했다.

1. **최적점이 어디인지** — 256 B. 미만은 정확히 2x 벌점, 초과는 완만히 악화.
2. **어디로 갈 수 있는지** — 분할 축이 2의 거듭제곱인 attn_out만. H=3840인 qkv·ffn은 막힘.
3. **과거의 해석이 틀렸다는 것** — V181의 이득은 런 길이가 아니라 inter-slice reduce 제거였고, 그 오해 위에 세운 후속 계획(V200/V202)은 취소됐다.

결과. attn_out을 256 B 런으로 옮기고(V198, warm -6.5% / cold -9.9%), ffn 선두의 store 2개를 1개로 합쳐(V199, -11,078, 6/6 잡 승) **Arena 6.229**. 공식 제출(V204)은 attn_out draw가 나빠 5.9032로 찍혔다.

이 캠페인의 교훈은 결과보다 방법에 있다. "한계에 도달했다"는 판단은 한계를 직접 재보기 전까지는 가설일 뿐이다. 서로 다른 것들이 같은 값에 모이면 물리 법칙처럼 보이지만, 공통의 실수도 똑같이 그렇게 보인다.

9.7 지금 서 있는 자리

공식 5.9032 / Arena 6.229. 차이는 코드가 아니라 공식 채점의 attn_out draw(56,845 vs Arena 47.9k~52.0k)다.

전송률로 보면 아직 절반이다.

커널	현재	최적점	도달률
qkv	314 B/cyc	618	51%
attn_out	315 B/cyc	618	51%
ffn	321 B/cyc	618	52%

열려 있는 것.

축	원시	내용
H=3840을 2의 거듭제곱으로 재배치	13	되면 qkv·ffn에도 256 B 런이 열린다. 남은 가장 큰 축이다
재제출로 좋은 draw 확보	—	코드 변경 없이 기대 +2%. 제출 예산 하루 5회
선두 명령 병합을 다른 커널에	14	값은 위치가 정한다(V201이 반례)

9.8 죽은 길 (다시 가지 말 것)

실험	왜 안 됐나
V8_weight_rows_interleaved_dma	가중치 행을 교차 배치했는데 DMA 노드가 전혀 안 움직였다 . 스케줄러의 DMA 비용 모델은 슬라이스 배치 순서를 안 본다
V21_qkv_rope_tables_early	rope 테이블 gather를 앞당겼는데 불변 . 스케줄러가 이미 hoist하고 있었다
V34_attnout_scale_in_rmsnorm	스케일을 rmsnorm으로 옮겼는데 완전히 중립
V39_x2_single_store	<code>StreamUnmatchedSegment</code> 컴파일 실패. <code>commit_view</code> 는 스트림 축과 tile 축이 구조적으로 같아야 통과한다
V42_attnout_tile_shapes	20/20/20 → 28/28/4로 바꿨더니 +1,042 . 스케줄러가 4행 타일을 두 번째로 로드해서 마지막 로드가 28행이 됐다
V15 ~ V41 의 HBM 사본	없는 축은 DMA write를 복제하지 않는다 . 26세대가 여기 오염됐다
V49 / V52 HBM 사본	사본을 제대로 채우면 store당 Core 명령 3개 × in-order 발행이 임계경로가 된다. makespan으로도 c=1을 못 이기고(V49), 실측으로도 +10k 였다(V52). 이 방향은 하드웨어에서 닫혔다
V167 / V168 출력 gather 후 store	패딩 레이아웃의 64-디스크립터 store가 실물에서 싸다 . ring-256 gather가 더 비싸서 둘 다 느려졌다
V200 / V202 ffn 청크 재분할	V181 이득의 원인을 잘못 짚어 세운 계획. ffn down은 weight 960 B-scale 120 B로 8청크가 이미 최적 (4청크는 weight 1920 B로 손해, 16청크는 scale 60 B로 손해)
디바이스 팬아웃 · DMA 병렬성 · 바이트 축소	<code>chip=1</code> 이 이미 8 PE를 쓰고, DmaEngine은 1개이며, 픽스처 가중치가 PRNG 난수라 압축이 안 된다. 전부 확인 후 닫음
V183 ffn down 세그먼트 통합	정확도는 PASS인데 v181 대비 +27k . 세그먼트 이득(≤19k)보다 x broadcast 480 패킷·f32 hop이 컸다
V137 / V138 qkv x를 f8 한 조각으로	정확도 FAIL (q 11.69% / k 27.53%, atol 0.04에 max Δ 0.113). f8e4m3 가수 3비트는 3,840항 내적에서 상쇄되지 않는다. 속도 이득도 없었다

구조적으로 배제된 것. × 영역(`host/` , `api/` , `axes.rs` , `tests/`) 최적화, `ops::sliding_attention` (Stage 1 대상 아님), `#[device]` 시그니처 변경, `ops*.rs` 이동.

틀체인 제약 (설계 단계에서 피할 것).

- fetch LUT는 연달아 못 건다**. `CanApplyFetchTableLookup` 이 첫 fetch 위치에서만 성립하고 `FetchCast<bf16>` for f8 도 없다. NVFP4를 bf16으로 만들려면 DM 사본을 반드시 한 번 거쳐야 한다 — V29가 f8×f8 경로를 택한 이유다.
- `commit_view` 는 스트림 축과 tile 축이 구조적으로 같아야 한다 (V39 실패).

10. 남은 열린 질문

1. $H=3840$ 을 2의 거듭제곱 조각으로 재배치할 수 있는가? 이게 남은 가장 큰 축이다. 되면 qkv-ffn에도 256 B 런이 열리고, 전송률 51% → 최적점 쪽으로 갈 여지가 생긴다. $3840 = 2^8 \times 15$ 라 단순 분할로는 안 되고, 15를 어딘가 다른 축으로 흡수시켜야 한다.
2. 618 B/cycle 위에는 무엇이 있는가? 그것도 스펙(1,500)의 41%다. 곡선의 최적점이 왜 41%에서 멈추는지는 아직 설명되지 않았다. HBM 스택 교대(V198의 원안)처럼 아직 안 재본 축이 있다.
3. 재제출로 draw를 개선할 것인가? 코드 변경 없이 공식 $5.90 \rightarrow 6.0$ 대가 기대된다. 제출 예산은 하루 5회다.
4. 선두 명령 병합을 qkv-`attn_out`에도 적용할 수 있는가? ffn에서는 -3.5%였고 qkv에서는 중립이었다. 위치가 값을 정하므로 각 커널의 임계경로를 따로 봐야 한다.
5. 상위권은 어디까지 갔는가? 우리가 5.9032로 올라선 시점의 직전 1위는 5.667이었다. 같은 곡선 앞에 있다면 경쟁은 "누가 $H=3840$ 문제를 먼저 푸는가"로 수렴한다.
6. (주최측 미확정) 제출 마감, 팀당 최대 제출 횟수의 정확한 정의.

11. 참고문헌

하드웨어 · 톨체인 (1차 자료)

- TCP: A Tensor Contraction Processor for AI Workloads — ISCA 2024, FuriosaAI. 슬라이드 · 발표 영상
- FuriosaAI RNGD: A Tensor Contraction Processor — IEEE Micro (Hot Chips 2024 특집). PDF
- FuriosaAI RNGD 스펙 — developer.furiosa.ai (BF16 256 / FP8 512 TFLOPS, INT8 512 / INT4 1024 TOPS, HBM3 48GB @ 1.5TB/s, SRAM 256MB, 5nm, 1.0GHz, 150W)
- Programming Tensor Contraction Processors — [furiosa-opt book](https://furiosa-opt-book.github.io). Quick Start (슬라이드당 DM 512 KB), Schedule, Diagnosis, Memory Performance, Kernel Optimizer, Schedule Viewer
- `furiosa_opt_std` API — docs.rs · `prelude` · `contraction` · `vector`
- FuriosaAI Arena — arena.furiosa.ai · `furiosa-arena-cli`

개념 배경 (§2를 더 파고 싶을 때)

- Williams, Waterman & Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. CACM 2009 — §2.6의 산술 강도·메모리 바운드 개념의 출처.
- Einstein summation / tensor contraction — NumPy `einsum` 문서가 §2.2를 손으로 만져보기에 제일 좋다.

모델 아키텍처

- Gemma Team. Gemma 3 Technical Report. [arXiv:2503.19786](https://arxiv.org/abs/2503.19786) — local/global 교대 sliding-window 어텐션, 128K+ 컨텍스트, 멀티모달. 이 저장소 40+8 레이어 구조의 직계 조상.
- Zhang & Sennrich. Root Mean Square Layer Normalization. [arXiv:1910.07467](https://arxiv.org/abs/1910.07467) — `shared/rmsnorm.rs` 가 구현하는 것.
- Shazeer. GLU Variants Improve Transformer. [arXiv:2002.05202](https://arxiv.org/abs/2002.05202) — `shared/mlp.rs` 의 GeGLU.
- Su et al. RoFormer: Enhanced Transformer with Rotary Position Embedding. [arXiv:2104.09864](https://arxiv.org/abs/2104.09864) — `sliding/rope.rs` ($\theta=10,000$), `full/rope.rs` ($\theta=1,000,000$).

- Ainslie et al. **GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.** [arXiv:2305.13245](#) — `Ns=8` KV 헤드에 `Gs=2` 쿼리가 붙는 구조.
- Milakov & Gimelshein. **Online normalizer calculation for softmax.** [arXiv:1805.02867](#) — `full/attention.rs` 의 페이지별 online softmax. Stage 2용.

양자화 (FFN의 NVFP4 경로)

- Elangovan et al. **LO-BCQ: Block Clustered Quantization for 4-bit (W4A4) LLM Inference.** [arXiv:2502.05376](#)
- Dettmers & Zettlemoyer. **The case for 4-bit precision: k-bit Inference Scaling Laws.** [arXiv:2212.09720](#)
- Blumenberg et al. **Improving Block-Wise LLM Quantization by 4-bit Block-Wise Optimal Float (BOF4).** [arXiv:2505.06653](#)

배경 (Stage 2 대비)

- Wang et al. **RATTENTION: Towards the Minimal Sliding Window Size in Local-Global Attention Models.** [arXiv:2506.15545](#)
- Cabannes et al. **Short window attention enables long-term memorization.** [arXiv:2509.24552](#)
- Alexandridis et al. **FLASH-D: FlashAttention with Hidden Softmax Division.** [arXiv:2505.14201](#)
- Chen et al. **INT-FlashAttention: Enabling Flash Attention for INT8 Quantization.** [arXiv:2409.16997](#)

저장소 내부 문서

[RULES.md](#) (최상위 규칙) · [RESULTS.md](#) (실험 기록) · [SOTA.md](#) (신기록 서사) · [README.md](#) (대회 규정 원문) · [OPTIMIZATION.md](#) (최적화 워크플로) · [ARCHITECTURE.md](#) (저장소 지도) · [COMPETITION_INFO.md](#) (공지 원문)

이 보고서의 헤드라인 수치는 공식 채점 결과(`moa-submitter 46bc9eed`)이고, Arena 실측은 별도로 표기했다. 그 외 실측은 Arena cold/warm 구분을 명시했다(\$8.4). `makespan`은 \$7.4와 \$9.3에서 실측과의 비교용으로만 등장한다. 최신 실험 기록은 [RESULTS.md](#), SOTA 서사는 [SOTA.md](#)에 있다.